

**PENERAPAN *ENTERPRISE RESOURCE
PLANNING* PADA *SINGLE BOARD COMPUTER***

Proposal Tugas Akhir

Oleh

**Eldwin Pradipta
18222042**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Juni 2026**

LEMBAR PENGESAHAN

PENERAPAN *ENTERPRISE RESOURCE PLANNING* PADA *SINGLE BOARD COMPUTER*

Proposal Tugas Akhir

Oleh

Eldwin Pradipta
18222042

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Proposal Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 9 Juni 2026

Pembimbing

Dr. Ir. Rinaldi, M.T.
NIP. 196512101994021001

DAFTAR ISI

DAFTAR GAMBAR	v
DAFTAR TABEL	vi
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	3
I.4 Batasan Masalah	3
I.5 Metodologi	4
II STUDI LITERATUR	6
II.1 Enterprise Resource Planning (ERP)	6
II.1.1 Definisi dan Evolusi ERP	6
II.1.2 Komponen dan Arsitektur ERP	7
II.1.3 Contoh-Contoh ERP	8
II.2 Proses Bisnis Lab Pengujian Lingkungan	9
II.3 Laboratory Information Management System (LIMS)	10
II.4 Arsitektur Prosesor dan <i>Single-Board Computer</i> (SBC)	11
II.4.1 Arsitektur Prosesor CISC/x86 vs RISC/ARM	11
II.4.2 <i>Single-Board Computer</i> (SBC): <i>Raspberry Pi</i> dan <i>Orange Pi</i>	11
II.5 Penelitian Terkait Development Aplikasi pada SBC	13
II.5.1 Penelitian Development <i>Web-Server</i> pada SBC	13
II.5.2 Deployment ERP pada SBC	14
II.6 Infrastruktur <i>Deployment</i> dan Arsitektur Sistem	15
II.6.1 <i>On-Premise</i>	15
II.6.2 <i>Cloud</i>	16
II.6.2.1 IaaS (Infrastructure-as-a-Service)	17
II.6.2.2 PaaS (Platform-as-a-Service)	17
II.6.2.3 SaaS (Software-as-a-Service)	17
II.6.3 <i>Hybrid</i>	18
II.6.4 Arsitektur Sistem: <i>Modular Monolith</i> vs <i>Microservice</i>	18
III ANALISIS MASALAH	20
III.1 Profil Perusahaan	20
III.2 Analisis Kondisi Saat Ini	20

III.2.1	Proses Operasional Manual	21
III.2.2	Upaya Digitalisasi Sebelumnya	22
III.3	Analisis Kebutuhan	23
III.3.1	Kebutuhan Fungsional	24
III.3.2	Kebutuhan Non-Fungsional	25
III.4	Justifikasi Pemilihan Teknologi	25
III.4.1	Pendekatan Pengembangan: <i>Custom</i> vs. <i>Off-the-Shelf</i>	25
III.4.2	Justifikasi <i>Framework</i> : Laravel dan Filament	26
III.4.3	Justifikasi Arsitektur: <i>Modular Monolith</i>	26
III.5	Justifikasi Pemilihan Perangkat Keras	27
III.5.1	Alternatif Perangkat Keras	27
III.5.1.1	Alternatif 1: Raspberry Pi 5	27
III.5.1.2	Alternatif 2: Orange Pi 5 Pro	27
III.5.1.3	Alternatif 3: Mini PC Intel N100	28
III.5.2	Justifikasi Pemilihan Orange Pi 5 Pro	28
IV	DESAIN KONSEP SOLUSI	29
IV.1	Gambaran Umum Arsitektur Sistem	29
IV.2	Alur Proses Bisnis Sistem (<i>To-Be</i>)	30
IV.2.1	Gambaran Umum Alur Operasional	30
IV.2.2	Alur <i>Sales</i> ke <i>Order</i>	31
IV.2.3	Alur <i>Order</i> ke Pengambilan Sampel	31
IV.2.4	Alur Pengambilan Sampel ke Penyimpanan	32
IV.2.5	Alur Penyimpanan ke Pengujian	32
IV.2.6	Manajemen Jadwal Pengujian	33
IV.3	Desain Data (<i>Entity Relationship Diagram</i>)	33
IV.4	Daftar <i>Use Case</i>	35
V	RENCANA SELANJUTNYA	37
V.1	Metodologi Pengembangan	37
V.1.1	Pendekatan Manajemen Proyek	37
V.1.2	Proses Pengembangan	38
V.1.3	Alat dan Infrastruktur Pengembangan	38
V.1.4	Peran AI dalam Pengembangan	38
V.2	Rencana Pengembangan	39
V.2.1	Jadwal Pengembangan	39
V.2.2	Detail Fase Pengembangan	39
V.2.2.1	Fase 0: Penyiapan Dokumentasi dan Infrastruktur (Minggu 1–2)	39

V.2.2.2	Fase 1: <i>Sales</i> ke <i>Order</i> (Minggu 3–4)	40
V.2.2.3	Fase 2: <i>Order</i> ke <i>Acquisition</i> (Minggu 5–7)	40
V.2.2.4	Fase 3: <i>Acquisition</i> ke <i>Storing</i> (Minggu 8–9)	40
V.2.2.5	Fase 4: <i>Storing</i> ke <i>Testing</i> (Minggu 10–11)	41
V.2.2.6	Fase 5: Manajemen Jadwal Pengujian (Minggu 12)	41
V.2.2.7	Fase 6: Penyeliaan (Minggu 13–14)	41
V.2.3	Catatan Jadwal	41
V.3	Analisis Risiko	42

DAFTAR GAMBAR

II.1	Aliran informasi dan material dalam model bisnis proses (Diadaptasi dari (Monk dan Wagner 2008))	8
II.2	Perbandingan antarmuka pengguna <i>Odoo</i> dan <i>ERPNext</i>	9
II.3	<i>Single-board computer</i> yang digunakan dalam penelitian (Sumber: (R. P. Ltd. 2024; Shenzhen Xunlong Software Co. 2024))	12
II.4	Topologi infrastruktur jaringan pada deployment berbasis <i>single-board computer</i>	14
II.5	Topologi infrastruktur jaringan pada deployment <i>on-premise</i>	16
II.6	Topologi infrastruktur jaringan pada deployment <i>cloud computing</i>	17
III.1	BPMN proses bisnis SSLab level 0 (kondisi saat ini)	21
III.3	Tampilan formulir <i>Quotation</i> pada aplikasi lama SSLab	23
III.4	Salah satu tampilan antarmuka aplikasi lama SSLab	23
IV.1	BPMN sistem Senling level 0 (<i>to-be</i>)	30
IV.2	BPMN alur <i>sales</i> ke <i>order</i> (<i>to-be</i>)	31
IV.3	BPMN alur <i>order</i> ke pengambilan sampel (<i>to-be</i>)	31
IV.4	BPMN alur pengambilan sampel ke penyimpanan (<i>to-be</i>)	32
IV.5	BPMN alur penyimpanan ke pengujian (<i>to-be</i>)	33
IV.6	BPMN manajemen jadwal pengujian (<i>to-be</i>)	33
IV.7	ERD sistem Senling	34
V.1	Jadwal pengembangan sistem Senling	39

DAFTAR TABEL

II.1	Perbandingan Arsitektur CISC/x86 dan RISC/ARM	11
II.2	Perbandingan utama Raspberry Pi 5 dan Orange Pi 5 Pro	13
III.1	Perbandingan Alternatif Perangkat Keras untuk Server Senling . . .	28
IV.1	Daftar <i>use case</i> sistem Senling	36
V.1	Analisis Risiko Implementasi Sistem ERP	42

BAB I

PENDAHULUAN

I.1 Latar Belakang

Sejak pertama kali komputer dikenalkan pada tahun 1960, berbagai perusahaan manufaktur membuat aplikasi untuk mencatat sumber daya material. Pada tahun 1970 *Materials Requirements Planning* (MRP) pertama kali dikenalkan, kemudian berkembang menjadi *MRP II* dengan sistem manajemen sumber daya material yang lebih komprehensif. Aplikasi ini kemudian berkembang dari pencatatan dan pengaturan menjadi alat bantu manajemen operasional yang disebut *Enterprise Resource Planning* (ERP). Dengan ERP, *enterprise* dapat melakukan manajemen keuangan, sumber daya manusia, hingga manajemen rantai pasokan dalam satu *platform*. Memasuki tahun 1990-an *market* ERP menjadi sangat kompetitif, namun setelah *dotcom bubble* pada 2001 hanya tersisa tiga *vendor* utama yaitu SAP, Oracle, dan Infor (Goldston 2020).

Pada awalnya, ERP merupakan aplikasi untuk mengatur sumber daya pada *enterprise* besar yang membutuhkan investasi awal sangat besar. Setelah meledaknya popularitas *cloud computing* yang memungkinkan model biaya berulang tanpa investasi infrastruktur, usaha mikro, kecil, dan menengah (UMKM) mulai tertarik (Goldston 2020). Keunggulan utama ERP terletak pada interoperabilitasnya: sistem dirancang mengikuti rantai pasok atau *value chain* sehingga seluruh proses bisnis terhubung dalam satu *platform* (Boza dkk. 2015). Fleksibilitas *deployment*—baik *cloud* maupun *on-premise*—membuat ERP semakin relevan lintas skala organisasi, termasuk munculnya *vendor* khusus UMKM seperti Odoo dan Mekari.

ERP dapat di-*deploy* menggunakan dua model infrastruktur utama: *cloud* dan *on-premise*, dengan alternatif ketiga *hybrid* yang belakangan mulai digunakan. Model *cloud* mencakup tiga jenis layanan—*Infrastructure-as-a-Service* (IaaS), *Platform-as-a-Service* (PaaS), dan *Software-as-a-Service* (SaaS)—yang masing-masing menawarkan tingkat abstraksi infrastruktur berbeda (Odun-Ayo dkk. July 2018). Model *on-premise* menempatkan ERP pada *server* milik organisasi sendiri: investasi awal lebih tinggi, namun tidak ada biaya berulang selain operasional

(Younus dkk. June 2024). Bagi organisasi dengan data sensitif atau keterbatasan koneksi internet, *on-premise* tetap menjadi pilihan yang lebih sesuai dibanding *cloud*.

Untuk menjalankan ERP *on-premise* dengan biaya kepemilikan yang rendah, diperlukan infrastruktur yang hemat biaya sekaligus hemat daya. Salah satu kandidat yang layak dipertimbangkan adalah perangkat berbasis prosesor *advanced RISC machine* (ARM). Dibanding arsitektur *x86* yang mendominasi *server* konvensional, ARM dirancang dengan orientasi efisiensi daya yang lebih tinggi (Blem, Menon, dan Sankaralingam February 2013; Aroca dan Gonçalves December 2012). Efisiensi ini menjadikan ARM dominan pada perangkat seperti *smartphone* dan *tablet*, dan kini mulai merambah ke *Single-Board Computer* (SBC) seperti Raspberry Pi dan Orange Pi sebagai *server* skala kecil.

Penerapan ERP *on-premise* di atas infrastruktur seperti ini paling relevan untuk domain dengan proses bisnis yang spesifik, data yang sensitif, dan skala pengguna yang terbatas — karakteristik yang dimiliki oleh lab pengujian lingkungan komersil.

Lab pengujian lingkungan komersil menjalankan proses bisnis yang khas, dari penawaran dan pengambilan sampel di lapangan hingga pengujian di laboratorium dan penerbitan sertifikat hasil uji. Pada lab skala kecil dan menengah, tahap-tahap ini kerap masih dijalankan secara manual menggunakan perkakas seperti *spreadsheet* tanpa integrasi antar tahap, sehingga koordinasi antar tim bergantung pada komunikasi informal yang rawan inkonsistensi. Upaya digitalisasi seadanya pun sering tidak berkelanjutan ketika dibangun tanpa desain sistem yang memadai. Di sisi lain, solusi ERP *off-the-shelf* seperti Odoo atau ERPNext kerap tidak sesuai karena alur pengujian yang terlalu spesifik untuk modul generik, sementara biaya lisensinya tidak sebanding dengan skala operasional lab. Kebutuhan yang muncul adalah sistem ERP yang dirancang khusus mengikuti proses bisnis lab, berjalan *on-premise* di infrastruktur hemat biaya.

Beberapa penelitian sebelumnya menguji SBC sebagai *server*, namun terbatas pada beban kerja sederhana seperti *web server* dengan *traffic* HTTP (Putra dan Sujana 2019)—belum representatif terhadap beban kerja ERP yang menuntut operasi *database* intensif dan interaksi *multi-user concurrent* (Maddodi, Jansen, dan Jong March 2018). SBC modern seperti Orange Pi menghadirkan kapasitas memori dan komputasi yang jauh lebih tinggi (Álvarez Ariza dan Baez January 2022), namun penggunaannya sebagai server ERP belum dieksplorasi. Yang lebih mendasar: belum ada implementasi sistem ERP *custom* untuk domain lab pengujian lingkungan komersil, apalagi yang berjalan *on-premise* di SBC. Gap inilah yang mendorong

dua pertanyaan penelitian: **bagaimana merancang sistem ERP *custom* yang sesuai dengan proses bisnis *end-to-end* lab pengujian lingkungan komersil?** dan **bagaimana metodologi pengembangan yang efektif untuk membangun sistem ERP *custom* pada domain tersebut?**

I.2 Rumusan Masalah

1. Bagaimana merancang sistem *enterprise resource planning custom* yang sesuai dengan proses bisnis *end-to-end* lab pengujian lingkungan komersil?
2. Bagaimana metodologi pengembangan yang efektif untuk membangun sistem ERP *custom* pada domain lab pengujian lingkungan?

I.3 Tujuan

1. Menghasilkan rancangan sistem ERP *custom* yang mencakup proses bisnis *end-to-end* lab pengujian lingkungan komersil, meliputi arsitektur sistem, desain modul, dan desain domain.
2. Mendokumentasikan metodologi pengembangan sistem ERP *custom* untuk domain lab pengujian lingkungan komersil.

I.4 Batasan Masalah

1. Sistem ERP yang dirancang dalam proposal ini ditujukan untuk *deployment on-premise* menggunakan Orange Pi 5 Pro sebagai *server* pengembangan dan *user acceptance testing* (UAT), tanpa perbandingan terhadap infrastruktur *cloud* atau *Virtual Private Server*.
2. Modul yang masuk dalam ruang lingkup perancangan meliputi tujuh modul: registrasi sampel, penjadwalan pengujian, manajemen hasil uji, pelaporan sertifikat hasil uji, inventori reagen dan alat, manajemen klien, serta modul keuangan yang mencakup *invoice* dan manajemen piutang/utang usaha.
3. Modul *human resource* dan *payroll*, serta fitur manajemen multi-cabang, tidak termasuk dalam ruang lingkup proposal ini.
4. UAT dengan pengguna nyata tidak masuk dalam ruang lingkup proposal ini karena data parameter pengujian aktual belum tersedia.

5. Ruang lingkup proposal ini dibatasi hingga tahap perancangan sistem—meliputi arsitektur, desain modul, dan desain domain—bukan implementasi final yang siap dioperasikan di lingkungan produksi.

I.5 Metodologi

1. Studi Literatur

Mengumpulkan referensi mengenai ERP, proses bisnis lab pengujian lingkungan, dan standar ISO/IEC 17025. Studi literatur juga mencakup kajian atas pilihan arsitektur sistem dan *tech stack* yang relevan untuk konteks pengembangan ini.

2. Analisis Kebutuhan

Mengidentifikasi proses bisnis *end-to-end* lab pengujian lingkungan komersil melalui observasi dan wawancara, kemudian mendokumentasikan kebutuhan fungsional dan non-fungsional dalam bentuk *use case* dan pemodelan proses bisnis.

3. Perancangan Sistem

Merancang arsitektur sistem, desain modul, dan desain domain berdasarkan kebutuhan yang telah diidentifikasi. Keluaran perancangan mencakup diagram BPMN *to-be*, *entity-relationship diagram* (ERD), dan spesifikasi *use case*.

4. Implementasi

Membangun sistem menggunakan Laravel 12 dan Filament 5 sebagai *framework* utama, PostgreSQL sebagai basis data, Redis sebagai *cache*, dan Nginx sebagai *web server*. Pengembangan mengikuti arsitektur *modular monolith* dengan pemisahan domain yang eksplisit.

5. Pengujian

Melakukan pengujian fungsionalitas per modul untuk memvalidasi kesesuaian implementasi dengan spesifikasi *use case*. Pengujian dilakukan pada Orange Pi 5 Pro sebagai lingkungan pengembangan dan UAT.

6. Evaluasi Hardware

Mengevaluasi kesesuaian Orange Pi 5 Pro sebagai *server* pengembangan dan UAT untuk sistem ERP *on-premise* skala lab, meliputi aspek kapasitas dan stabilitas operasional.

BAB II

STUDI LITERATUR

II.1 Enterprise Resource Planning (ERP)

II.1.1 Definisi dan Evolusi ERP

Enterprise resource planning (ERP) adalah sistem informasi yang didesain untuk integrasi dan optimisasi proses bisnis dan transaksi di sebuah *enterprise* dengan mencakup area berbeda seperti operasional, sumber daya manusia (SDM), keuangan/akutansi, dan penjualan (Boza dkk. 2015). Implementasi ERP yang sukses dapat meningkatkan efisiensi operasional dan mendukung pengambilan keputusan strategis di *enterprise* besar (Boza dkk. 2015), dengan studi terkini menunjukkan adopsi yang meningkat pada segmen usaha kecil dan menengah (Hasanah, Saputra, dan Hilyatin 2024).

Keuntungan penggunaan ERP mencakup tiga kategori utama: keuntungan teknologi (keunggulan kompetitif, peningkatan kinerja finansial, dan kemitraan rantai pasokan), keuntungan *knowledge sharing* (pertukaran informasi antar unit dan kolaborasi departemen), serta keuntungan kepemimpinan (keterlibatan manajemen dan komunikasi antar kelompok bisnis) (Goldston 2020).

Sebelum ERP menjadi sistem informasi yang kita kenal sekarang, sistem informasi ini mengalami evolusi dari aplikasi kontrol inventori sederhana hingga menjadi alat bantu mengambil keputusan dan automasi proses bisnis terintegrasi. Pada era 1960-an, organisasi mulai memperkenalkan komputer untuk melacak inventaris dan mengatur produksi—yang kemudian berkembang menjadi *Materials Requirements Planning* (MRP) pada dekade 1970-an dengan kemampuan pembelian, peramalan, dan penjadwalan (Goldston 2020). Integrasi lebih luas terjadi pada 1980-an dengan MRP-II yang menambahkan fitur perencanaan lanjutan, diikuti pada 1990-an ketika Gartner Group secara resmi menciptakan istilah *Enterprise Resource Planning* (ERP) untuk sistem yang mengintegrasikan seluruh operasional dan akuntansi perusahaan. Konsolidasi pasar terjadi pasca gelembung *dotcom* tahun 2001, menyisakan tiga vendor raksasa—Oracle, SAP, dan Infor—yang mendominasi hingga sekarang. Perkembangan krusial berikutnya adalah adopsi *cloud com-*

puting yang memindahkan tanggung jawab infrastruktur ke vendor dan mengurangi overhead IT; pada 2016, implementasi ERP berbasis *cloud* meningkat 40 persen dibanding tahun sebelumnya, menandai pergeseran paradigma deployment yang relevan untuk konteks penelitian ini (Goldston 2020).

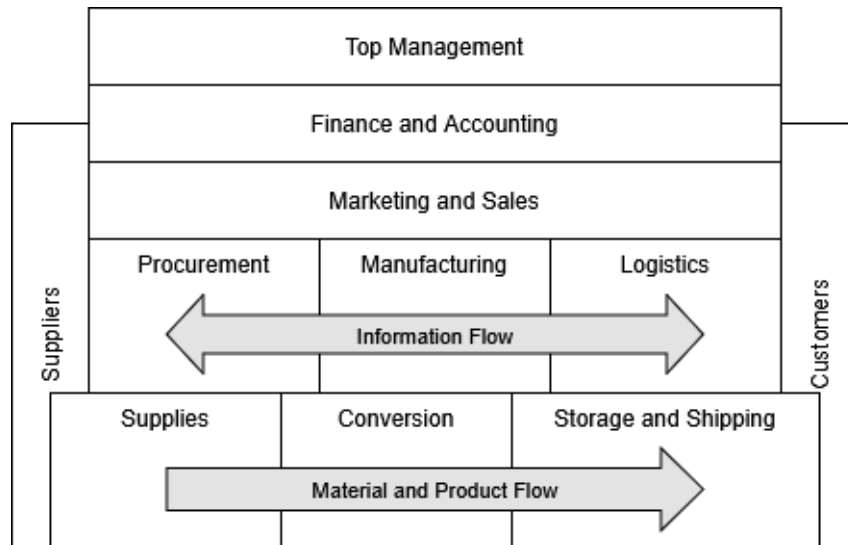
II.1.2 Komponen dan Arsitektur ERP

Seiring perkembangan teknologi komputasi, sistem informasi perusahaan mengalami evolusi dari arsitektur *mainframe* menjadi *client-server*. Dalam arsitektur *client-server* yang diadopsi oleh sistem ERP modern, beban kerja komputasi didistribusikan antara server pusat dan komputer pengguna (*client*) yang berfungsi sebagai antarmuka. Server bertanggung jawab atas pengelolaan basis data dan eksekusi logika aplikasi yang kompleks, sementara klien menyediakan tampilan dan interaksi pengguna (Monk dan Wagner 2008). Model ini meningkatkan *scalability*, memungkinkan perusahaan untuk menambah kapasitas seiring pertumbuhan kebutuhan bisnis tanpa harus mengganti seluruh infrastruktur.

Komponen fungsional ERP tersusun atas berbagai modul terintegrasi yang dirancang untuk menangani area bisnis spesifik. Modul-modul ini meliputi, namun tidak terbatas pada, *Sales and Distribution* (SD), *Materials Management* (MM), *Production Planning* (PP), *Financial Accounting* (FI), dan *Human Resources* (HR), yang kesemuanya beroperasi di atas satu basis data terpusat (Monk dan Wagner 2008). Integrasi data melalui basis data terpusat ini menjadi karakteristik fundamental yang membedakan ERP dari sistem informasi tradisional berbasis *silos*. Setiap transaksi yang dimasukkan di satu modul secara otomatis memperbarui data terkait di modul lain, menciptakan *single source of truth* dan mengeliminasi redundansi data (Monk dan Wagner 2008).

Integrasi yang mendalam ini memungkinkan aliran informasi dan material yang efisien di seluruh organisasi, dari pemasok hingga pelanggan. Diagram pada Gambar II.1 mengilustrasikan bagaimana sistem ERP memfasilitasi aliran informasi horizontal antar fungsi seperti *Procurement*, *Manufacturing*, dan *Logistics*, berbeda dengan model bisnis fungsional yang terpisah-pisah.

Sebagai tulang punggung sistem ERP, infrastruktur basis data harus memenuhi persyaratan kinerja yang tinggi, menjaga konsistensi data, dan mendukung transaksi konkuren dari berbagai modul secara bersamaan. Basis data ERP harus mampu menangani volume data yang besar dengan waktu respons yang cepat, menjamin integritas referensial antar tabel, serta menyediakan mekanisme *backup* dan *recovery* untuk keandalan sistem. Menurut standar kualitas perangkat lunak ISO/IEC 25010, sistem ERP harus memenuhi karakteristik efisiensi kinerja (*per-*



Gambar II.1 Aliran informasi dan material dalam model bisnis proses (Diadaptasi dari (Monk dan Wagner 2008))

formance efficiency) yang mencakup *time behaviour* (waktu respons dan *throughput* yang memadai), *resource utilization* (penggunaan CPU, memori, dan *storage* yang efisien), serta *capacity* (kemampuan menangani volume data dan pengguna sesuai spesifikasi) (International Organization for Standardization 2011). Dalam perkembangan modern, arsitektur ERP juga mengadopsi pendekatan terdistribusi dan berorientasi layanan (*service-oriented architecture/SOA*) yang memungkinkan fleksibilitas *deployment* dan integrasi dengan sistem eksternal melalui antarmuka pemrograman aplikasi (API).

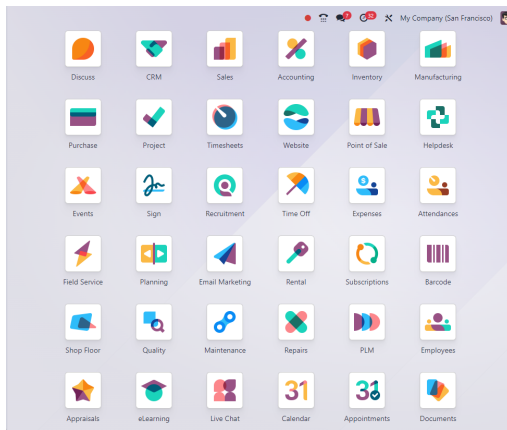
II.1.3 Contoh-Contoh ERP

Seperti yang telah dijelaskan sebelumnya, *vendor* ERP komersial seperti Oracle, SAP, dan Infor mendominasi pasar *enterprise* besar dengan sistem yang matang dan kaya fitur (Goldston 2020). Namun, biaya implementasi yang sangat tinggi, lisensi mahal berbasis *per user*, kompleksitas implementasi yang membutuhkan tim ahli khusus, dan risiko *vendor lock-in* membuat solusi komersial kurang sesuai untuk organisasi skala kecil-menengah.

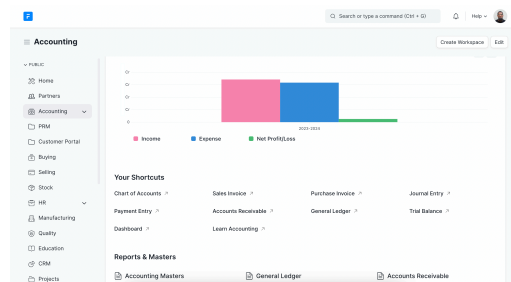
Sebagai alternatif yang lebih terjangkau, sistem ERP *open-source* muncul dengan keuntungan yang menarik. Tidak ada biaya lisensi, *source code* tersedia untuk modifikasi, komunitas pengguna yang aktif menyediakan dukungan, dan arsitektur yang modular memungkinkan organisasi mengimplementasikan hanya modul yang relevan dengan kebutuhan bisnis mereka. Pendekatan modular ini penting untuk organisasi yang tidak memerlukan fungsionalitas *enterprise* penuh

tetapi membutuhkan sistem yang dapat berkembang seiring pertumbuhan bisnis (Boza dkk. 2015). Dengan menghilangkan hambatan biaya dan meningkatkan fleksibilitas, solusi *open-source* membuka akses ERP bagi organisasi yang sebelumnya tidak memiliki sumber daya untuk digitalisasi.

Di antara berbagai ERP *open-source* yang tersedia, *Odoo* dan *ERPNext* adalah yang paling matang dan banyak diadopsi. *Odoo* merupakan ERP *open-source* paling populer dengan komunitas global yang besar, ekosistem aplikasi yang luas melalui *Odoo App Store*, dan dokumentasi yang detail (S.A. 2025). Arsitektur modular *Odoo* memungkinkan organisasi memilih dan mengintegrasikan modul sesuai kebutuhan, menjadikannya fleksibel untuk berbagai skala bisnis. *ERPNext* dikembangkan di atas *framework* Frappe menggunakan Python, dengan antarmuka intuitif dan implementasi yang lebih ringan (F. T. P. Ltd. 2024). *ERPNext* mengedepankan kesederhanaan tanpa mengorbankan fungsionalitas inti dengan arsitektur berbasis Python/Frappe yang memudahkan kustomisasi.



(a) Antarmuka dashboard aplikasi *Odoo*
(Sumber: Wikipedia)



(b) Antarmuka modul akuntansi *ERPNext*
(Sumber: Wikipedia)

Gambar II.2 Perbandingan antarmuka pengguna *Odoo* dan *ERPNext*

Selain keduanya, terdapat alternatif lain seperti Apache OFBiz dan Dolibarr. *Odoo* dan *ERPNext* relevan sebagai referensi dalam penelitian ini karena keduanya menunjukkan cakupan fungsional ERP yang mapan—dari manajemen klien, inventori, hingga keuangan—yang menjadi acuan dalam menentukan cakupan modul Senling, sistem ERP *custom* yang dirancang dalam penelitian ini.

II.2 Proses Bisnis Lab Pengujian Lingkungan

Lab pengujian lingkungan komersil menjalankan siklus layanan yang dimulai dari penerimaan permintaan klien dan berakhir pada penerbitan sertifikat hasil uji. Stan-

dar ISO/IEC 17025 menetapkan persyaratan kompetensi untuk laboratorium pengujian dan kalibrasi, mencakup persyaratan manajemen (pengendalian dokumen, penanganan pengaduan, ketidaksesuaian) dan persyaratan teknis (personel, peralatan, metode pengujian, keterlacakan pengukuran) (International Organization for Standardization 2005). Laboratorium yang terakreditasi standar ini wajib memastikan seluruh operasionalnya terdokumentasi dan dapat ditelusuri sesuai ketentuan yang berlaku.

Alur proses bisnis lab pengujian lingkungan komersil umumnya mencakup lima tahap yang saling bergantung. Tahap pertama adalah penawaran kepada klien—penetapan parameter pengujian, jadwal, dan harga berdasarkan permintaan. Tahap kedua adalah penjadwalan dan pelaksanaan pengambilan sampel di lokasi klien, yang memerlukan koordinasi antara petugas lapangan, klien, dan laboratorium. Tahap ketiga adalah pengujian sampel di laboratorium sesuai metode referensi yang berlaku, bergantung pada ketersediaan peralatan, reagen, dan personel kompeten. Tahap keempat adalah verifikasi dan penerbitan sertifikat hasil uji dalam format yang memenuhi ketentuan akreditasi. Tahap kelima adalah penagihan dan pencatatan penerimaan pembayaran.

Setiap tahap menghasilkan dan membutuhkan data yang saling terkait: data klien, data sampel, jadwal, hasil pengujian, stok reagen, dan transaksi keuangan. Tanpa sistem yang mengintegrasikan semua ini, koordinasi antar tahap bergantung pada komunikasi manual yang rawan inkonsistensi. Sistem ERP yang dirancang untuk lab pengujian lingkungan harus mampu memodelkan alur ini secara *end-to-end* dan memastikan data dari satu tahap tersedia untuk tahap berikutnya tanpa intervensi manual.

II.3 Laboratory Information Management System (LIMS)

Laboratory Information Management System (LIMS) adalah perangkat lunak yang dirancang untuk mengelola data dan alur kerja laboratorium: penerimaan sampel, pelacakan status pengujian, pencatatan hasil, pengelolaan instrumen, dan pemeliharaan rantai keterlacakan data (*chain of custody*). LIMS berfokus pada aspek ilmiah dan teknis operasional lab—memastikan integritas data dari sumber sampel hingga laporan hasil uji.

Hubungan LIMS dengan ERP bersifat komplementer. LIMS mengelola kedalaman data pengujian, sementara ERP mengelola proses bisnis yang melingkupinya—penjualan, keuangan, inventori, dan manajemen klien. Lab komersil skala besar sering mengoperasikan keduanya secara terpisah dan mengintegrasikannya melalui API. Untuk lab skala kecil-menengah, pemisahan ini

menambah kompleksitas implementasi dan biaya operasional yang tidak sebanding dengan skala yang ada.

Senling bukan LIMS. Sistem ini dirancang sebagai ERP *custom* yang mencakup seluruh proses bisnis lab secara *end-to-end*, dengan modul manajemen sampel dan hasil uji sebagai bagian dari domain operasional yang lebih besar. Pendekatan satu sistem terpadu dipilih karena lab pengujian lingkungan komersil skala kecil-menengah membutuhkan integrasi antara operasional lab, manajemen klien, inventori, dan keuangan dalam satu platform—bukan dua sistem terpisah yang perlu diintegrasikan.

II.4 Arsitektur Prosesor dan *Single-Board Computer* (SBC)

II.4.1 Arsitektur Prosesor CISC/x86 vs RISC/ARM

Arsitektur prosesor menentukan cara unit pemrosesan mengeksekusi instruksi dan berdampak langsung pada efisiensi daya perangkat. Dua paradigma utama adalah *Complex Instruction Set Computer* (CISC) yang dominan pada arsitektur x86 untuk desktop dan server, dan *Reduced Instruction Set Computer* (RISC) yang identik dengan ARM pada perangkat hemat daya (Aroca dan Gonçalves December 2012). Meskipun studi benchmark menunjukkan bahwa efisiensi energi lebih bergantung pada optimasi mikroarsitektur daripada tipe *instruction set* semata (Blem, Menon, dan Sankaralingam February 2013), ARM unggul dalam konsumsi daya karena instruksi sederhana dengan panjang tetap memerlukan sirkuit yang lebih ringkas. Karakteristik ini menjadikan ARM pilihan dominan untuk SBC—perangkat yang beroperasi dalam batas konsumsi daya rendah sekaligus menangani beban kerja server skala kecil.

Tabel II.1 Perbandingan Arsitektur CISC/x86 dan RISC/ARM

Karakteristik	CISC/x86	RISC/ARM
Desain Instruksi	Variabel, kompleks	Tetap, sederhana
Eksekusi	Multi-siklus/out-of-order	Single-siklus/in-order
Konsumsi Daya	Cenderung tinggi	Lebih rendah
Aplikasi Utama	PC, Server	Smartphone, Tablet, SBC
Upgrade Kinerja	Optimalisasi mikroarsitektur	Efisiensi dan integrasi SoC

II.4.2 *Single-Board Computer* (SBC): *Raspberry Pi* dan *Orange Pi*

Single-board computer (SBC) adalah komputer utuh yang seluruh komponen utamanya—prosesor, memori, penyimpanan, dan port I/O—terintegrasikan pada

satu papan sirkuit. Konsep SBC mulai dikenal luas sejak era 1970-an dengan munculnya produk seperti KIM-1 dan Apple I, yang menawarkan solusi komputasi sederhana dan terjangkau untuk keperluan pendidikan dan hobi elektronika. Namun, perkembangan pesat SBC terjadi pada dekade 2010-an berkat inovasi teknologi dan kebutuhan terhadap perangkat hemat daya serta ukuran ringkas yang mendukung aplikasi *embedded* dan *edge computing*.

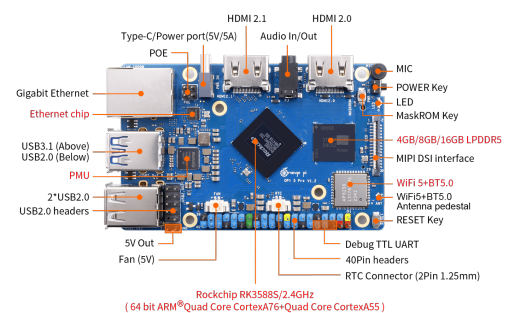
Seiring waktu, SBC berevolusi dari sekadar perangkat pembelajaran menuju platform fleksibel—dari robotika hingga *digital signage* dan otomasi industri. Faktor harga ekonomis, komunitas pengguna yang berkembang, serta ekosistem perangkat lunak yang aktif membuat SBC semakin diminati oleh pengembang yang membutuhkan *prototyping* cepat dengan biaya rendah. Dua contoh SBC yang paling banyak digunakan adalah *Raspberry Pi* dan *Orange Pi*.

Raspberry Pi 5 merupakan generasi terbaru dari keluarga *Raspberry Pi* yang menawarkan peningkatan substansial dalam kinerja, konektivitas, dan dukungan multimedia. Produk ini dilengkapi prosesor quad-core ARM Cortex-A76 2.4GHz, RAM hingga 8GB, dual HDMI 4K, PCIe standar, serta berbagai port baru yang mendukung aplikasi AI, otomasi, dan multimedia terbaru (R. P. Ltd. 2024).

Orange Pi 5 Pro adalah SBC rilisan terbaru dari Xunlong yang membidik segmen *embedded computing* kinerja tinggi dengan harga terjangkau. Menggunakan prosesor *octa-core* Rockchip RK3588S hingga 2.4GHz dan RAM LPDDR4/LPDDR5 hingga 32GB, *Orange Pi 5 Pro* mendukung *output* video 8K, slot M.2 NVMe SSD, serta berbagai fitur konektivitas lanjutan (Shenzhen Xunlong Software Co. 2024).



(a) *Raspberry Pi 5*



(b) *Orange Pi 5 Pro*

Gambar II.3 *Single-board computer* yang digunakan dalam penelitian (Sumber: (R. P. Ltd. 2024; Shenzhen Xunlong Software Co. 2024))

Raspberry Pi 5 dan *Orange Pi 5 Pro* dapat dibandingkan dari sisi spesifikasi utama berikut:

Tabel II.2 Perbandingan utama Raspberry Pi 5 dan Orange Pi 5 Pro

Spesifikasi	Raspberry Pi 5	Orange Pi 5 Pro
Prosesor	Cortex-A76 Quad-core 2.4GHz	RK3588S Octa-core 2.4GHz
RAM Maksimal	8GB LPDDR4X	32GB LPDDR4/LPDDR5
Output Video	Dual HDMI 4K	Output 8K HDMI, DP
Storage	microSD, PCIe M.2 SSD	microSD, M.2 NVMe SSD
Port Ekspansi	PCIe, GPIO, CSI/DSI	PCIe, GPIO, M.2, DSI/CSI
Harga Pasaran	Rp1,200,000–1,800,000	Rp1,800,000–2,700,000

Berdasarkan spesifikasi di atas, kedua SBC berada dalam kelas perangkat yang mampu menjalankan aplikasi bisnis skala kecil. *Raspberry Pi 5* menawarkan stabilitas ekosistem dan dukungan komunitas yang matang, sementara *Orange Pi 5 Pro* memberikan konfigurasi memori lebih besar (hingga 16GB) dengan harga yang lebih terjangkau. Perbandingan lebih lanjut dan justifikasi pemilihan *Orange Pi 5 Pro* sebagai server *on-premise* Senling dibahas pada Bab III.

II.5 Penelitian Terkait Development Aplikasi pada SBC

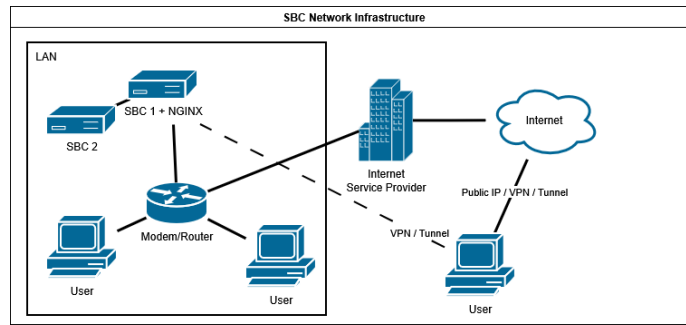
II.5.1 Penelitian Development *Web-Server* pada SBC

Perkembangan *single-board computer* (SBC) seperti *Raspberry Pi* membuka kemungkinan baru untuk pengembangan aplikasi berbasis server dengan biaya investasi yang rendah dan konsumsi daya minimal. Penelitian oleh Putra dkk. menunjukkan bahwa SBC dapat membangun *cluster server* berbasis *load balancing* (Putra dan Sujana 2019). Melalui pengujian HTTP dengan *traffic* statis, *cluster* dengan metode *round robin* mempercepat waktu respons hingga 59% dibandingkan konfigurasi server tunggal, sekaligus memberikan *redundancy* sehingga sistem tetap berjalan meskipun salah satu *node* terganggu. Hasil ini menempatkan SBC sebagai platform server yang memadai untuk beban kerja ringan.

Sementara itu, penelitian oleh Arief dkk. membangun dan menguji *portable server* berbasis SBC untuk mendukung pembelajaran praktis di ruang kelas atau laboratorium (Arief dkk. September 2018). Sistem tersebut portabel secara fisik, berbiaya rendah, dapat diakses dari beragam perangkat, dan tidak bergantung pada koneksi internet eksternal. Pengujian menunjukkan server SBC siap digunakan kurang dari tiga menit sejak *booting* dan mampu melayani *login* beberapa perangkat secara simultan, dengan konsumsi CPU, memori, dan *bandwidth* jaringan yang rendah pada kondisi akses minimum.

Kedua penelitian memiliki fokus berbeda: Putra dkk. mengevaluasi kinerja dan *reliability* arsitektur *cluster server* untuk layanan jaringan, sedangkan Arief

dkk. menekankan *usability* dan efisiensi sumber daya dalam konteks pembelajaran. Keduanya menyoroti keunggulan SBC yang sama—fleksibilitas *deployment*, kemudahan konfigurasi, dan konsumsi daya yang rendah—yang relevan untuk aplikasi skala kecil hingga menengah. Pada studi Putra dkk., penggunaan *tool automation* seperti Ansible juga menyederhanakan manajemen sistem melalui orkestrasi tanpa *agent* (Putra dan Sujana 2019).



Gambar II.4 Topologi infrastruktur jaringan pada deployment berbasis *single-board computer*

Sebagaimana diilustrasikan pada Gambar II.4, topologi infrastruktur berbasis SBC umumnya dikonfigurasi sebagai server mandiri (*standalone*) atau *cluster* kecil yang beroperasi di tepi jaringan (*edge*). Konfigurasi ini memungkinkan *deployment* aplikasi server yang portabel dan independen dari infrastruktur pusat data tradisional, sesuai untuk kebutuhan lokal atau terdistribusi.

Namun, kedua penelitian terbatas pada kompleksitas aplikasi yang diuji. Studi Putra dkk. menggunakan *traffic* HTTP sederhana yang belum representatif terhadap beban kerja seperti ERP, yang menuntut transaksi basis data, pemrosesan *business logic*, dan interaksi banyak *user* secara konkuren. Studi Arief dkk. menempatkan SBC sebagai server pembelajaran dengan akses terbatas dan beban *baseline* minimum, sehingga belum mengeksplorasi batas kapasitas maupun skenario *stress test multi-user* yang kompleks.

II.5.2 Deployment ERP pada SBC

Kemajuan teknologi *single-board computer* (SBC) seperti *Raspberry Pi* dan *Orange Pi* membuka peluang untuk mengimplementasikan ERP yang sebelumnya terbatas pada server konvensional. Implementasi penuh ERP pada SBC telah dibuktikan melalui berbagai eksperimen komunitas, meskipun dokumentasi resmi *Odoo* maupun *ERPNext* masih terbatas pada forum diskusi dan blog tutorial (Cottafavi 2023; Peppe8o 2023; Forum 2023).

Tutorial oleh Cottafavi memaparkan tahapan instalasi dan konfigurasi *Odoo* pada *Raspberry Pi*, termasuk pengaturan basis data PostgreSQL dan aktivasi beberapa modul bisnis (Cottafavi 2023). Artikel Peppe8o menyajikan implementasi *Odoo* pada *Raspberry Pi* secara *end-to-end* untuk skala riset maupun bisnis mikro (Peppe8o 2023). Forum Frappe mendokumentasikan pengalaman mendirikan *ERP-Next* secara penuh pada *Raspberry Pi 5*, sekaligus tantangan nyatanya: keterbatasan perangkat ARM, kebutuhan dependensi, dan kinerja basis data (Forum 2023).

Keterbatasan sumber daya seperti RAM, prosesor, dan *storage* membuat ERP pada SBC lebih sesuai untuk skala kecil dengan pengguna terbatas. Pengalaman ini menunjukkan SBC cukup memadai sebagai server ERP skala lab, sehingga relevan sebagai platform pengembangan untuk sistem ERP *custom* seperti Senling (Cottafavi 2023; Peppe8o 2023; Forum 2023).

II.6 Infrastruktur *Deployment* dan Arsitektur Sistem

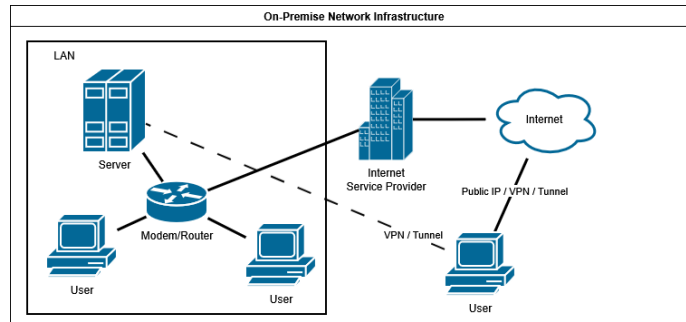
Pemilihan model *deployment* infrastruktur berdampak langsung pada kontrol sistem, keamanan data, dan skalabilitas aplikasi *enterprise* seperti ERP. Model *deployment* menentukan bagaimana infrastruktur komputasi, penyimpanan, dan jaringan diatur, dikelola, dan diakses oleh organisasi (Younus dkk. June 2024). Keputusan ini bergantung pada faktor-faktor seperti anggaran infrastruktur, kapabilitas teknis tim operasional, dan persyaratan keamanan data. Secara umum, terdapat tiga model utama yang mendominasi: *on-premise*, *cloud*, dan *hybrid*—masing-masing dengan karakteristik dan pertimbangan yang berbeda (Odun-Ayo dkk. July 2018). Di samping model *deployment*, pemilihan arsitektur sistem menentukan bagaimana komponen perangkat lunak diorganisasi dan di-*deploy*—keputusan yang berdampak pada kompleksitas pengembangan dan kemudahan pemeliharaan jangka panjang.

II.6.1 *On-Premise*

Model *on-premise* merujuk pada *deployment* infrastruktur secara internal di lokasi fisik yang dimiliki atau disewa oleh organisasi, di mana seluruh perangkat keras (*hardware*), perangkat lunak (*software*), dan manajemen sistem menjadi tanggung jawab penuh organisasi tersebut (Younus dkk. June 2024). Dalam model ini, organisasi harus menginvestasikan modal awal yang besar untuk membeli *server*, perangkat jaringan, sistem penyimpanan, lisensi perangkat lunak, serta membangun atau menyewa ruang *data center* dengan infrastruktur pendukung seperti pendingin udara dan *uninterruptible power supply* (UPS). Organisasi juga bertanggung jawab penuh atas instalasi, konfigurasi, pemeliharaan, pembaruan sistem, pencadangan

data (*backup*), dan perencanaan pemulihan bencana (*disaster recovery*).

Keuntungan utama model ini terletak pada kontrol penuh terhadap data, konfigurasi sistem, dan jadwal pembaruan yang memungkinkan organisasi menyesuaikan infrastruktur dengan kebutuhan spesifik tanpa terikat pada batasan penyedia layanan eksternal. Keamanan fisik dapat dijaga langsung, dan *uptime* sistem tidak bergantung pada koneksi internet.



Gambar II.5 Topologi infrastruktur jaringan pada deployment *on-premise*

Secara topologi jaringan sebagaimana diilustrasikan pada Gambar II.5, seluruh komponen infrastruktur—mulai dari server aplikasi hingga basis data—berada dalam jaringan lokal (LAN) yang terisolasi dari jaringan publik. Akses pengguna dilakukan langsung melalui infrastruktur internal, yang menjamin latensi rendah dan keamanan data karena trafik tidak melintasi internet publik. Investasi awal yang dibutuhkan cukup besar, dan tanggung jawab penuh atas pemeliharaan sistem memerlukan kapabilitas teknis internal (Younus dkk. June 2024).

II.6.2 Cloud

Konsep *cloud computing* mengubah paradigma *deployment* infrastruktur dengan menawarkan akses *on-demand* ke *shared pool* sumber daya komputasi melalui internet, menghilangkan kebutuhan organisasi untuk memiliki dan mengelola perangkat keras secara langsung (Odun-Ayo dkk. July 2018). Model ini dibangun di atas lima karakteristik fundamental: *on-demand self-service*, *broad network access*, *resource pooling*, *rapid elasticity*, dan *measured service* yang menyediakan transparansi penggunaan untuk penagihan berbasis konsumsi aktual (Odun-Ayo dkk. July 2018; Younus dkk. June 2024). Sejak diperkenalkan oleh Salesforce pada 1999 dengan model SaaS, diikuti Amazon Web Services (AWS) pada 2006 dengan layanan IaaS, *cloud computing* telah berkembang menjadi ekosistem kompetitif yang melahirkan tiga model layanan utama: *Infrastructure-as-a-Service* (IaaS), *Platform-as-a-Service* (PaaS), dan *Software-as-a-Service* (SaaS).

II.6.2.1 IaaS (Infrastructure-as-a-Service)

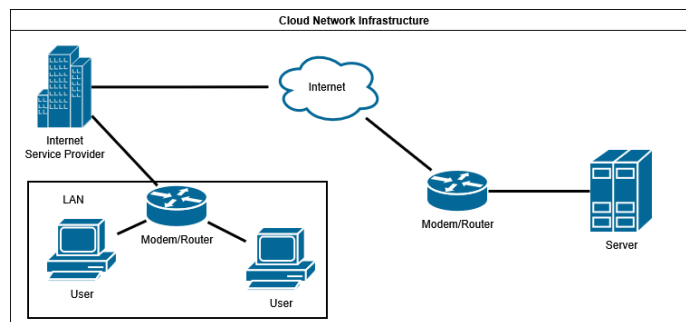
Model IaaS menyediakan infrastruktur komputasi fundamental berupa *server virtual*, penyimpanan, dan jaringan yang disewa sesuai kebutuhan, di mana penyedia layanan mengelola perangkat keras dan virtualisasi sementara pengguna memiliki kontrol penuh atas sistem operasi dan aplikasi (Odun-Ayo dkk. July 2018; Younus dkk. June 2024).

II.6.2.2 PaaS (Platform-as-a-Service)

PaaS menyediakan platform pengembangan lengkap yang mencakup sistem operasi, basis data, dan *development tools*, memungkinkan pengembang fokus pada logika aplikasi tanpa mengelola infrastruktur yang mendasarinya (Odun-Ayo dkk. July 2018; Younus dkk. June 2024).

II.6.2.3 SaaS (Software-as-a-Service)

SaaS menyediakan aplikasi perangkat lunak yang di-*hosting* dan diakses melalui internet tanpa perlu instalasi atau pemeliharaan lokal, di mana penyedia layanan bertanggung jawab penuh atas seluruh *stack* teknologi termasuk pembaruan dan keamanan (Odun-Ayo dkk. July 2018; Younus dkk. June 2024).



Gambar II.6 Topologi infrastruktur jaringan pada deployment *cloud computing*

Berbeda dengan model *on-premise*, topologi jaringan *cloud* seperti ditunjukkan pada Gambar II.6 menempatkan seluruh sumber daya komputasi di pusat data penyedia layanan yang diakses melalui internet publik. Pengguna tidak lagi terhubung ke server lokal, melainkan melalui gerbang internet yang aman, memungkinkan aksesibilitas global namun menciptakan ketergantungan kritis pada konektivitas jaringan eksternal. Untuk konteks ERP, implementasi SaaS seperti *Odoo Online* dan *ERPNext Cloud* menyediakan kemudahan *deployment* minimal dengan arsitektur *multi-tenancy* dan skalabilitas otomatis, namun dengan keterbatasan kustomisasi mendalam dibandingkan *deployment on-premise* atau IaaS.

Meskipun *cloud computing* menawarkan fleksibilitas dan kemudahan deployment, model ini memiliki keterbatasan yang relevan untuk konteks lab pengujian lingkungan komersil. Biaya langganan bersifat berkelanjutan dan meningkat seiring penambahan modul atau pengguna (Younus dkk. June 2024). Ketergantungan pada koneksi internet stabil menjadi kendala di lokasi dengan infrastruktur jaringan terbatas. Untuk lab dengan data hasil uji yang bersifat *confidential*, model *on-premise* memberi kontrol penuh yang tidak tersedia pada layanan *cloud*.

II.6.3 *Hybrid*

Model *hybrid cloud* menggabungkan infrastruktur *private* dan *public cloud*, memungkinkan organisasi mengalokasikan beban kerja secara strategis berdasarkan sensitivitas data dan kebutuhan skalabilitas (Odun-Ayo dkk. July 2018). Keuntungan model ini mencakup fleksibilitas dalam alokasi sumber daya, optimisasi biaya dengan menjalankan beban kerja stabil di infrastruktur *private* sambil memanfaatkan *public cloud* untuk kebutuhan sementara, serta kemampuan menjaga kontrol tinggi untuk data sensitif sambil tetap menikmati skalabilitas layanan *public cloud* (Younus dkk. June 2024). Namun, kompleksitas manajemen infrastruktur tersebar memerlukan keahlian teknis tinggi dan *tools* orkestrasi canggih, menjadikan model ini lebih sesuai untuk organisasi dengan tim IT berkapasitas tinggi. Dalam konteks proposal ini, model *hybrid* tidak relevan karena tidak ada infrastruktur *cloud* eksisting yang perlu diintegrasikan dan kebutuhan adalah *deployment on-premise* penuh.

II.6.4 **Arsitektur Sistem: *Modular Monolith* vs *Microservice***

Pemilihan arsitektur sistem berdampak pada kompleksitas pengembangan, kemudahan *deployment*, dan kemampuan pemeliharaan jangka panjang. Dua pendekatan yang umum dipertimbangkan untuk sistem ERP skala menengah adalah arsitektur *microservice* dan arsitektur *modular monolith*.

Arsitektur *microservice* memecah sistem menjadi layanan-layanan kecil yang independen, masing-masing bertanggung jawab atas satu domain fungsional dan dapat di-*deploy* secara terpisah (Fowler dan Lewis 2014). Isolasi kegagalan, kemampuan *scaling* per layanan, dan fleksibilitas pemilihan teknologi per domain adalah keuntungan utamanya. Di sisi lain, pendekatan ini memerlukan infrastruktur koordinasi yang tidak sepele: *service discovery*, orkestrasi kontainer, *distributed tracing*, manajemen konfigurasi terdistribusi, dan penanganan kegagalan jaringan antar layanan. Overhead ini baru terbayar pada skala di mana beban tiap layanan

berbeda signifikan dan tim pengembang cukup besar untuk mengelola layanan secara paralel.

Arsitektur *modular monolith* mempertahankan satu *codebase* dan satu unit *deployment*, namun membagi kode ke dalam modul atau domain yang terisolasi secara logis. Komunikasi antar modul terjadi melalui *function call* dalam proses yang sama—lebih cepat dan lebih mudah di-*debug* dibanding *network call*. Pemisahan domain dijaga melalui konvensi arsitektur dan desain basis data, tanpa infrastruktur koordinasi antar layanan. *Deployment* lebih sederhana karena hanya ada satu artefak yang dikelola.

Skala operasional lab pengujian lingkungan komersil yang kecil—jumlah pengguna terbatas, volume transaksi tidak tinggi, tim pengembang kecil—membuat overhead arsitektur *microservice* tidak sebanding dengan manfaatnya. Senling mengadopsi *modular monolith* dengan pemisahan domain eksplisit yang tercermin dalam desain ERD: domain Customer, Product Knowledge, Sales, dan Operations membentuk kluster entitas yang terisolasi dalam satu basis data PostgreSQL. Pendekatan ini memberikan *maintainability* yang memadai pada tingkat desain, sekaligus kesederhanaan *deployment* yang sesuai untuk server *on-premise* berspesifikasi terbatas.

BAB III

ANALISIS MASALAH

III.1 Profil Perusahaan

Sentral Sistem Laboratory merupakan salah satu unit usaha di bawah PT Sentral Teknologi Managemen, perusahaan yang berdiri sejak tahun 1999 (Sentral Sistem Laboratory 2024). Lini pertamanya, Sentral Sistem Consulting, bergerak di bidang konsultansi manajemen dengan cakupan yang luas: kinerja bisnis, mutu, lingkungan, keselamatan kerja, energi, hingga pengembangan sumber daya manusia dan budaya perusahaan.

Bidang usaha kedua menyusul pada tahun 2014 melalui Sentral Sistem Calibration. Unit ini menyediakan jasa kalibrasi alat ukur dan pelatihan kalibrasi, dengan sebagian lingkup layanannya telah terakreditasi Komite Akreditasi Nasional (KAN).

Unit pengujian laboratorium baru berdiri belakangan, yaitu pada tahun 2025, dengan nama Sentral Sistem Laboratory (SSLab). Unit ini berfokus pada pengujian mekanik dan lingkungan serta melayani klien dari berbagai sektor industri. Sebagai lini termuda dalam kelompok usaha, SSLab menjadi objek pengembangan sistem ERP yang dibahas dalam proposal ini.

Ketiga unit tersebut menempatkan Sentral Sistem Group pada rantai layanan yang berurutan: konsultansi sistem manajemen, kalibrasi alat ukur, dan pengujian laboratorium. Posisi SSLab sebagai unit termuda turut menjelaskan kondisi yang dibahas pada bagian berikutnya, yaitu proses operasional yang belum sepenuhnya didukung sistem informasi terintegrasi.

III.2 Analisis Kondisi Saat Ini

Seluruh proses operasional SSLab—from penerimaan penawaran, penjadwalan pengujian, pencatatan hasil uji, hingga penerbitan sertifikat dan pencatatan transaksi keuangan—dijalankan tanpa sistem informasi yang terintegrasi. Kondisi ini terbentuk dari dua lapisan masalah: proses yang sepenuhnya manual, dan upaya

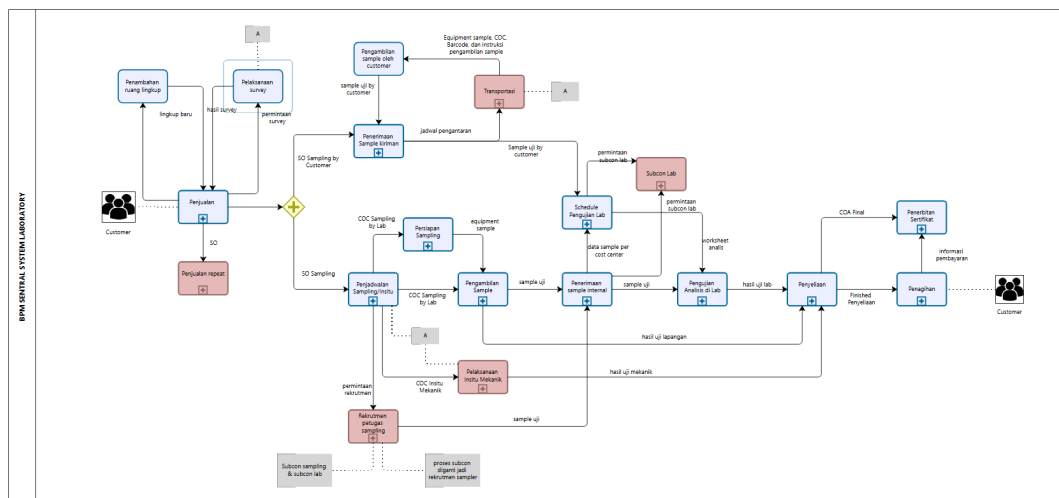
digitalisasi sebelumnya yang tidak berhasil diteruskan.

III.2.1 Proses Operasional Manual

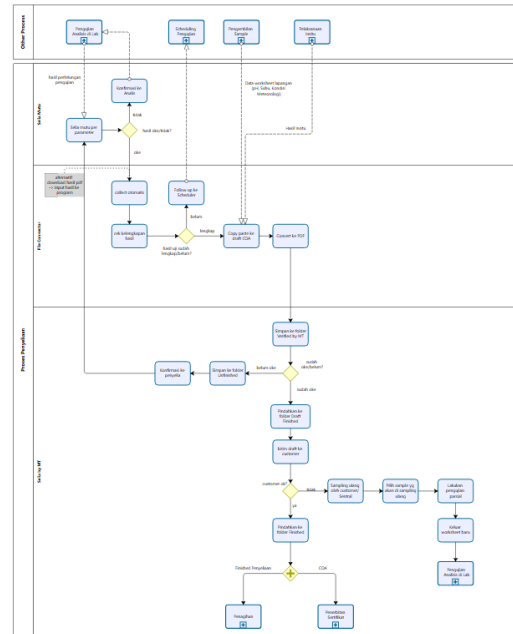
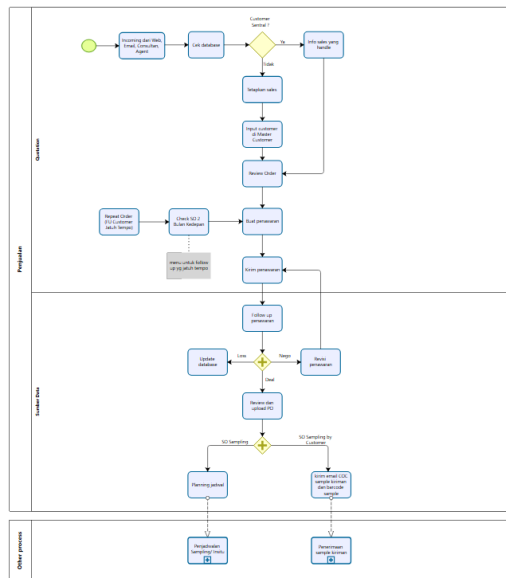
Seluruh alur operasional SSLab bertumpu pada Google Sheets yang tidak terhubung satu sama lain. Tim *sales* mencatat penawaran di dokumen terpisah dari tim operasional yang menjadwalkan pengujian, dan tim laboratorium yang mencatat hasil uji menggunakan berkas tersendiri pula. Setiap perubahan—revisi penawaran, pergeseran jadwal, atau koreksi hasil uji—memerlukan penyalinan manual antardokumen. Tidak ada validasi otomatis maupun jejak perubahan; konsistensi data bergantung sepenuhnya pada ketelitian individu.

Kondisi ini juga tidak *scalable*. Ketika volume pekerjaan bertambah, jumlah *spreadsheet* yang harus dikelola bertambah pula tanpa mekanisme agregasi. Pelaporan ke klien, termasuk penerbitan sertifikat hasil uji, memerlukan rekap manual dari beberapa sumber berbeda setiap kali. SSLab telah terakreditasi KAN (ISO/IEC 17025) (International Organization for Standardization 2005) untuk sebagian parameter pengujiannya, dan akreditasi tersebut antara lain mensyaratkan keterlacakan proses dan konsistensi rekaman—keduanya sulit dipenuhi secara konsisten dengan skema *spreadsheet* yang terpisah.

Diagram BPMN berikut menggambarkan kondisi proses bisnis SSLab sebelum sistem terintegrasi dikembangkan. Gambar III.1 menunjukkan gambaran umum alur dari penerimaan *order* hingga penerbitan sertifikat. Gambar III.2a dan III.2b memperlihatkan proses penjualan dan penyeliaan secara lebih rinci. Kedua diagram menunjukkan titik-titik keputusan yang tidak terdefinisi dengan jelas dan *exit* yang ambigu—gambaran proses yang dijalankan secara informal tanpa panduan sistem.



Gambar III.1 BPMN proses bisnis SSLab level 0 (kondisi saat ini)



Kode Sampling	Lokasi Sampling	Titik Sampling	Frekuensi	Produk Uji	Regulasi	Biaya Pengujian	Action
25A18-1-2	area1	1	2 C	UDARA AMBIEN	PPRI No. 22 tahun 2011 Lamp VI	2,200,000	[Edit] [Delete]
25A18-2-2	area1	2	2 C	UDARA AMBIEN	PPRI No. 22 tahun 2011 Lamp VI	2,200,000	[Edit] [Delete]
25A18-3-2	area2	1	3 C	UDARA AMBIEN	PPRI No. 22 tahun 2011 Lamp VI	2,200,000	[Edit] [Delete]
25A18-4-2	area2	2	3 C	UDARA AMBIEN	PPRI No. 22 tahun 2011 Lamp VI	2,200,000	[Edit] [Delete]

Gambar III.3 Tampilan formulir *Quotation* pada aplikasi lama SSLab

Menu Master Reference Method

1. Data yang diinput tidak sesuai dengan data headernya

Data yg diinput:

Data di header:

#	Reference Method	Judul Reference	Action
1	TEST	TEST2222	[Edit] [Delete]
2	TEST	TEST2222	[Edit] [Delete]
3	TEST3	TEST3	[Edit] [Delete]
4	TEST 23	ABCDEFH	[Edit] [Delete]
5	UDARA AMBIEN - BAGIAN 7: CARA UJI KADAR SULFUR DIOKSIDA (SO2) DENGAN METODE PARAROSANILIN MENGGUNAKAN SPEKTROFOTOMETER	SNI 7119-7:2017	[Edit] [Delete]

Gambar III.4 Salah satu tampilan antarmuka aplikasi lama SSLab

Ketiadaan desain yang terdokumentasi, cakupan yang parsial, dan infrastruktur yang tidak memadai membuat aplikasi lama tidak layak untuk dilanjutkan. Berbeda dari pendekatan tersebut, Senling dirancang dari analisis proses bisnis, penyusunan *use case*, dan pemodelan data, dengan cakupan seluruh alur operasional SSLab dari penerimaan *order* hingga pencatatan keuangan. Dua kondisi di atas—proses manual yang tidak terintegrasi dan upaya digitalisasi yang tidak berhasil—membentuk dasar analisis kebutuhan pada bagian berikutnya.

III.3 Analisis Kebutuhan

Dari kondisi saat ini, dua kelompok kebutuhan perlu dipenuhi: kebutuhan fungsional yang menentukan cakupan modul, dan kebutuhan non-fungsional yang memba-

tasi desain teknis sistem.

III.3.1 Kebutuhan Fungsional

Tujuh modul fungsional Senling dirancang untuk mencakup seluruh alur operasional SSLab dari penerimaan *order* hingga pencatatan keuangan. Setiap modul mengelola domain yang berbeda namun saling terintegrasi melalui basis data yang sama.

- a) **KF-01 — Manajemen Klien:** Sistem harus dapat menyimpan dan mengelola data klien, termasuk informasi kontak dan alamat pengambilan sampel. Modul ini menjadi titik awal alur operasional karena setiap *order* pengujian berasal dari klien yang terdaftar.
- b) **KF-02 — Registrasi Sampel:** Sistem harus dapat mencatat sampel yang masuk, baik yang dikirim oleh klien maupun yang diambil langsung oleh tim lapangan SSLab. Setiap sampel terhubung ke *order* klien dan parameter pengujian yang diminta.
- c) **KF-03 — Penjadwalan Pengujian:** Sistem harus dapat menjadwalkan proses pengujian berdasarkan ketersediaan analisis, alat, dan tenggat waktu. Penjadwalan yang terdokumentasi menggantikan koordinasi manual yang saat ini dilakukan melalui *spreadsheet* terpisah.
- d) **KF-04 — Manajemen Hasil Uji:** Sistem harus dapat mencatat hasil pengujian per parameter yang diuji oleh analisis. Hasil ini menjadi dasar penerbitan sertifikat dan harus terlacak hingga ke analisis yang bertanggung jawab.
- e) **KF-05 — Pelaporan Sertifikat Hasil Uji:** Sistem harus dapat menghasilkan dokumen sertifikat hasil uji berdasarkan data pengujian yang tercatat. Sertifikat merupakan dokumen resmi yang diterima klien sebagai bukti hasil pengujian laboratorium.
- f) **KF-06 — Inventori Reagen dan Alat:** Sistem harus dapat mengelola stok reagen dan peralatan laboratorium, termasuk kartu stok yang mencatat setiap penerimaan dan penggunaan. Inventori yang tercatat memungkinkan SSLab mendeteksi kebutuhan pengadaan sebelum stok habis.
- g) **KF-07 — Modul Keuangan:** Sistem harus dapat mencatat transaksi keuangan yang berkaitan dengan *order* klien, mencakup pembuatan *invoice*, pencatatan piutang (*accounts receivable*), dan pencatatan hutang (*accounts payable*).

III.3.2 Kebutuhan Non-Fungsional

Di luar cakupan fungsional, ada empat kebutuhan non-fungsional yang membatasi pilihan teknologi dan infrastruktur.

- a) **KNF-01 — Performa:** Sistem harus mampu melayani hingga 20 pengguna bersamaan (*concurrent users*) tanpa degradasi performa yang signifikan. Angka ini didasarkan pada estimasi beban puncak operasional SSLab dalam satu waktu.
- b) **KNF-02 — Deployment *On-Premise*:** Sistem harus dapat di-*deploy* sepenuhnya pada infrastruktur milik SSLab tanpa ketergantungan pada layanan pihak ketiga. Kebutuhan ini berasal dari keinginan SSLab untuk mengelola sendiri seluruh infrastruktur sistem, termasuk data, konfigurasi, dan pemeliharaan.
- c) **KNF-03 — Kompatibilitas dengan Orange Pi 5 Pro:** Sistem harus dapat berjalan pada Orange Pi 5 Pro (prosesor ARM RK3588S *octa-core*, RAM 16 GB, konsumsi daya 10–15W). Batasan ini mendorong efisiensi penggunaan sumber daya agar sistem tetap responsif di atas perangkat dengan spesifikasi terbatas dibanding *server* konvensional.
- d) **KNF-04 — *Maintainability*:** Arsitektur sistem harus modular sehingga modul baru dapat ditambahkan tanpa merombak komponen yang sudah ada. Kebutuhan ini relevan karena pengembangan dilakukan secara bertahap dan cakupan modul direncanakan berkembang setelah tahap desain ini selesai.

Kebutuhan-kebutuhan ini menentukan pilihan teknologi dan infrastruktur yang dibahas pada bagian berikutnya.

III.4 Justifikasi Pemilihan Teknologi

III.4.1 Pendekatan Pengembangan: *Custom* vs. *Off-the-Shelf*

Dua platform ERP *off-the-shelf* yang umum dipertimbangkan untuk organisasi skala kecil-menengah adalah Odoo dan SAP. Odoo tersedia dalam versi *open-source* dengan modul-modul generik yang mencakup CRM, inventori, akuntansi, dan manufaktur (S.A. 2025). SAP merupakan platform ERP kelas *enterprise* dengan cakupan fungsional yang luas, namun biaya lisensi dan implementasinya tidak proporsional untuk organisasi dengan skala SSLab.

Odoo menawarkan fleksibilitas kustomisasi dan komunitas yang besar, namun modulnya dirancang untuk proses bisnis generik. Proses operasional lab pengujian lingkungan—registrasi sampel berdasarkan parameter spesifik, penjadwalan analisis per metode uji, hingga penerbitan sertifikat hasil uji—tidak dapat dipetakan langsung ke modul standar Odoo tanpa kustomisasi mendalam pada level domain. Kustomisasi sejauh itu, dalam praktiknya, memerlukan upaya yang setara dengan membangun sistem dari nol, dengan tambahan beban memahami arsitektur *framework* yang sudah ada.

Atas dasar ini, pengembangan *custom* dipilih. Dengan membangun Senling dari nol, seluruh model domain, alur kerja, dan antarmuka dirancang sesuai proses bisnis SSLab tanpa kompromi terhadap keterbatasan modul generik.

III.4.2 Justifikasi *Framework*: Laravel dan Filament

Laravel dipilih sebagai *framework* utama karena tim pengembang sudah terbiasa dengan PHP. Familiaritas ini mengurangi waktu *onboarding* dan risiko teknis yang muncul saat mengadopsi ekosistem baru. Dibandingkan *framework* PHP lain seperti CodeIgniter—yang digunakan pada aplikasi lama SSLab—Laravel menawarkan ekosistem yang lebih *mature*: ORM ekspresif (Eloquent), sistem migrasi basis data, manajemen dependensi via Composer, dan konvensi yang mendorong struktur kode yang konsisten.

Filament dipilih sebagai lapisan panel admin di atas Laravel karena keterbatasan waktu pengembangan. Membangun antarmuka admin—tabel data, form, CRUD, manajemen RBAC—dari nol menghabiskan waktu yang signifikan sebelum logika domain bisnis dapat disentuh. Filament menyediakan komponen-komponen ini secara deklaratif. Waktu pengembangan dapat difokuskan pada logika domain lab yang spesifik. Kombinasi Laravel dan Filament membentuk fondasi *modular monolith* yang dijabarkan pada subbagian berikutnya.

III.4.3 Justifikasi Arsitektur: *Modular Monolith*

Dua pilihan arsitektur yang dipertimbangkan adalah *modular monolith* dan *microservice*. Arsitektur *microservice* memecah sistem menjadi layanan-layanan kecil yang berjalan dan di-*deploy* secara independen (Fowler dan Lewis 2014). Pendekatan ini menguntungkan pada sistem berskala besar atau dengan kebutuhan *scaling* per komponen, namun membawa *overhead* koordinasi yang nyata: setiap interaksi antar layanan melewati jaringan, *service discovery* perlu dikonfigurasi, dan penelusuran masalah melibatkan *distributed tracing*.

Skala operasional SSLab tidak memerlukan kompleksitas tersebut. Jumlah pengguna terbatas, volume transaksi tidak tinggi, dan tim pengembang kecil—kondisi di mana *overhead microservice* tidak sebanding dengan manfaatnya. *Modular monolith* dipilih karena memberikan keuntungan pemisahan domain sekaligus kesederhanaan *deployment*: satu *codebase*, satu proses, satu basis data PostgreSQL yang berjalan pada Orange Pi. Pemisahan domain dalam Senling terlihat eksplisit di struktur data: Customer, Product Knowledge, Sales, Operations, dan Finance masing-masing membentuk klaster entitas yang terisolasi dalam satu *deployment* yang sama.

III.5 Justifikasi Pemilihan Perangkat Keras

III.5.1 Alternatif Perangkat Keras

Pemilihan perangkat keras didasarkan pada kebutuhan SSLab untuk menjalankan Senling secara *on-premise* dengan anggaran yang terbatas. Tiga alternatif dipertimbangkan: Raspberry Pi 5, Orange Pi 5 Pro, dan Mini PC berbasis Intel N100.

III.5.1.1 Alternatif 1: Raspberry Pi 5

Raspberry Pi 5 merupakan generasi terbaru dari SBC paling populer di dunia, menggunakan prosesor *quad-core* ARM Cortex-A76 2.4GHz dan tersedia dalam varian RAM 4 GB dan 8 GB (R. P. Ltd. 2024). Konsumsi daya sistem berkisar 8–12W. Ekosistem Raspberry Pi memiliki komunitas global yang besar dengan dokumentasi yang *mature* dan dukungan distribusi Linux yang luas. Keterbatasan utamanya terletak pada kapasitas RAM maksimum 8 GB, yang menjadi batasan untuk menjalankan *stack* lengkap Senling (Nginx, PHP-FPM, PostgreSQL, Redis) secara bersamaan dengan beban pengguna puncak.

III.5.1.2 Alternatif 2: Orange Pi 5 Pro

Orange Pi 5 Pro berbasis prosesor Rockchip RK3588S dengan konfigurasi *octa-core* (4 core ARM Cortex-A76 2.4GHz + 4 core ARM Cortex-A55 1.8GHz) dalam arsitektur *big.LITTLE* (Shenzhen Xunlong Software Co. 2024). Perangkat ini tersedia hingga varian RAM 16 GB dan mendukung Ubuntu, Debian, serta Armbian. Konsumsi daya sistem berkisar 10–15W. Dibandingkan Raspberry Pi 5, Orange Pi 5 Pro menawarkan jumlah core yang lebih banyak dan kapasitas RAM dua kali lebih besar pada varian tertinggi. Harga unit termasuk casing dan adapter berkisar Rp 2 juta di pasar Indonesia (Shenzhen Xunlong Software Co. 2024).

III.5.1.3 Alternatif 3: Mini PC Intel N100

Mini PC berbasis Intel N100 sering dipertimbangkan sebagai alternatif *server* berdaya rendah berbasis x86. Meskipun TDP tercatat 6W, konsumsi daya aktual saat menjalankan beban transaksional dapat mencapai 25W akibat *power burst* pada level SoC (Blem, Menon, dan Sankaralingam February 2013). Mini PC N100 umumnya dijual dalam kondisi *barebone* tanpa RAM, sehingga biaya total pengadaan lebih tinggi dari SBC dengan memori terintegrasi. Arsitektur x86 juga tidak memiliki keunggulan khusus untuk *workload* seperti Senling dibandingkan ARM pada kisaran daya yang setara.

III.5.2 Justifikasi Pemilihan Orange Pi 5 Pro

Tabel III.1 merangkum perbandingan ketiganya.

Tabel III.1 Perbandingan Alternatif Perangkat Keras untuk Server Senling

Kriteria	RPi 5	OPi 5 Pro	Mini PC N100
Arsitektur ARM	✓	✓	×
RAM tersedia ≥ 16 GB	×	✓	✓
Konsumsi daya ≤ 15 W	✓	✓	×
Harga all-in $\leq$ Rp 3 juta	✓	✓	×

Orange Pi 5 Pro dipilih sebagai server *on-premise* untuk Senling. Raspberry Pi 5 gugur pada satu kriteria: kapasitas RAM maksimum 8 GB tidak cukup untuk menjalankan seluruh komponen *stack* Senling secara bersamaan dengan beban 20 pengguna puncak (KNF-01 dan KNF-03). Mini PC Intel N100 gugur pada dua kriteria: konsumsi daya aktual melampaui 15W dan harga total setelah penambahan RAM melebihi anggaran yang tersedia. Orange Pi 5 Pro memenuhi seluruh kriteria—ARM *octa-core*, RAM hingga 16 GB, konsumsi daya 10–15W, dan harga sekitar Rp 2 juta termasuk casing dan adapter (Shenzhen Xunlong Software Co. 2024).

BAB IV

DESAIN KONSEP SOLUSI

IV.1 Gambaran Umum Arsitektur Sistem

Senling adalah sistem ERP *custom* yang dirancang khusus untuk proses bisnis SSLab, dibangun dari awal menggunakan pendekatan *modular monolith*. Satu basis kode dan satu *deployment* mencakup seluruh fungsionalitas sistem — dari pengelolaan klien dan penawaran hingga operasional pengujian dan pelaporan hasil uji. Domain bisnis dipisahkan secara eksplisit dalam struktur kode maupun skema basis data, membentuk batas tanggung jawab yang jelas antar modul tanpa memerlukan layanan terpisah.

Permintaan dari pengguna masuk melalui Nginx sebagai *web server* dan *reverse proxy*, yang meneruskannya ke PHP-FPM sebagai *application server*. PHP-FPM menjalankan aplikasi Laravel 12 + Filament 5 (PHP 8.4) yang menangani seluruh logika bisnis dan antarmuka pengguna berbasis panel admin. Data disimpan di PostgreSQL sebagai basis data utama, sementara Redis menangani *caching* sesi dan data yang membutuhkan akses berulang dengan latensi rendah. Pengendalian akses diimplementasikan melalui Spatie Permission dan Filament Shield, yang mengatur *role-based access control* (RBAC) sesuai peran pengguna — Sales, Planning, Petugas Sampling, Laboran, Penyelia, dan Administrator.

Seluruh komponen berjalan dalam satu *deployment* di Orange Pi 5 Pro, sebuah *single board computer* yang ditempatkan secara fisik di SSLab. Tidak ada komponen operasional inti yang bergantung pada layanan eksternal; koneksi internet hanya digunakan untuk fitur pengiriman dokumen opsional ke klien. Pendekatan ini memenuhi kebutuhan SSLab untuk *self-hosting* penuh — mengelola data dan infrastruktur sendiri tanpa ketergantungan pada pihak ketiga.

Pemilihan arsitektur *modular monolith* didasarkan pada skala operasional SSLab yang kecil. Pengguna aktif terbatas pada tim internal lab dengan volume transaksi yang tidak memerlukan distribusi pemrosesan. Arsitektur terdistribusi membawa *overhead* koordinasi — *network calls* antar layanan, *service discovery*, dan kerumitan operasional — yang tidak sebanding dengan manfaatnya pada skala

ini. *Modular monolith* memberikan keuntungan pemisahan domain secara logis sekaligus kesederhanaan *deployment* yang sesuai untuk lingkungan *on-premise* di Orange Pi.

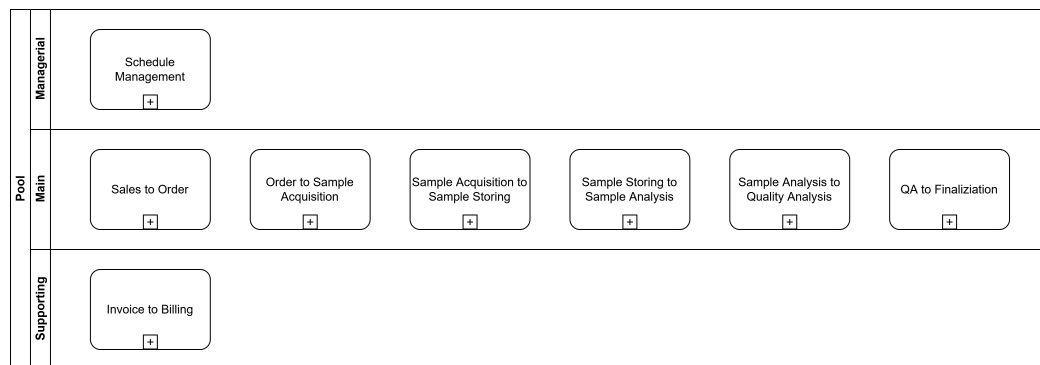
Domain bisnis Senling terbagi menjadi lima klaster: Customer, Product Knowledge, Sales, Operations, dan Finance. Setiap domain memiliki gugus entitas yang terisolasi dalam skema basis data, dengan interaksi antar domain hanya melalui relasi yang terdefinisi secara eksplisit. Pemisahan ini tampak pada ERD sistem (Gambar IV.7) dan menentukan batas tanggung jawab tiap modul fungsional yang dijabarkan pada bagian-bagian berikutnya.

IV.2 Alur Proses Bisnis Sistem (*To-Be*)

Bagian ini menggambarkan alur proses bisnis SSLab setelah sistem Senling diimplementasikan, mencakup enam alur utama dari penerimaan *order* hingga manajemen jadwal pengujian.

IV.2.1 Gambaran Umum Alur Operasional

Gambar IV.1 menggambarkan keseluruhan alur operasional Senling pada level 0, dari penerimaan *order* klien hingga penerbitan sertifikat hasil uji.

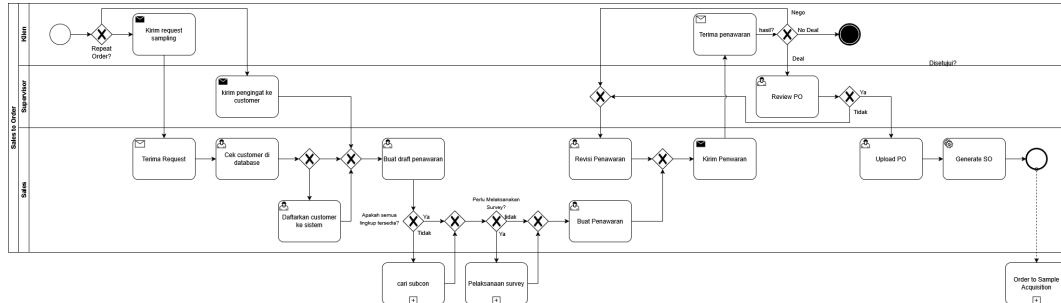


Gambar IV.1 BPMN sistem Senling level 0 (*to-be*)

Senling mengintegrasikan enam alur utama yang membentuk siklus operasional SSLab dari awal hingga akhir: penerimaan permintaan klien, perencanaan sampling, pelaksanaan sampling, penerimaan sampel di laboratorium, pengujian parameter, serta penyeliaan dan penerbitan sertifikat hasil uji (*Certificate of Analysis*). Setiap alur ditangani oleh modul yang berbeda, namun seluruhnya terhubung dalam satu basis data PostgreSQL. Status dan hasil dari satu tahap diteruskan ke tahap berikutnya melalui relasi entitas yang terdefinisi — tidak ada transfer data manual antar tahap.

IV.2.2 Alur Sales ke Order

Gambar IV.2 menggambarkan alur dari penerimaan permintaan klien hingga konfirmasi *order* pengujian dalam sistem Senling.

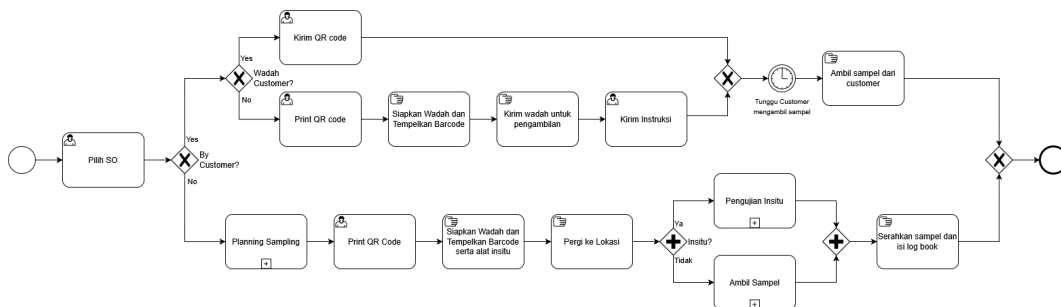


Gambar IV.2 BPMN alur *sales* ke *order* (to-be)

Alur ini dikelola oleh aktor Sales. Proses dimulai dengan verifikasi identitas klien di basis data — jika belum terdaftar, Sales memasukkan data klien baru (UC-03). Sales kemudian menyusun *draft* penawaran (UC-04) dan menerbitkan penawaran final (UC-05) yang memuat produk uji, parameter pengujian, dan biaya. Jika klien meminta penyesuaian, sistem menyimpan versi revisi sebagai baris baru tanpa menghapus versi sebelumnya (UC-06), sehingga riwayat negosiasi tetap terlacak. Setelah klien menyetujui penawaran dan mengunggah *Purchase Order* (UC-07), sistem secara otomatis membuat *Sales Order* beserta seluruh detail, daftar sampel, dan parameter pengujian melalui *QuotationService* dalam satu operasi atomik.

IV.2.3 Alur Order ke Pengambilan Sampel

Gambar IV.3 menggambarkan alur dari *order* yang telah dikonfirmasi hingga proses pengambilan atau penerimaan sampel di laboratorium.



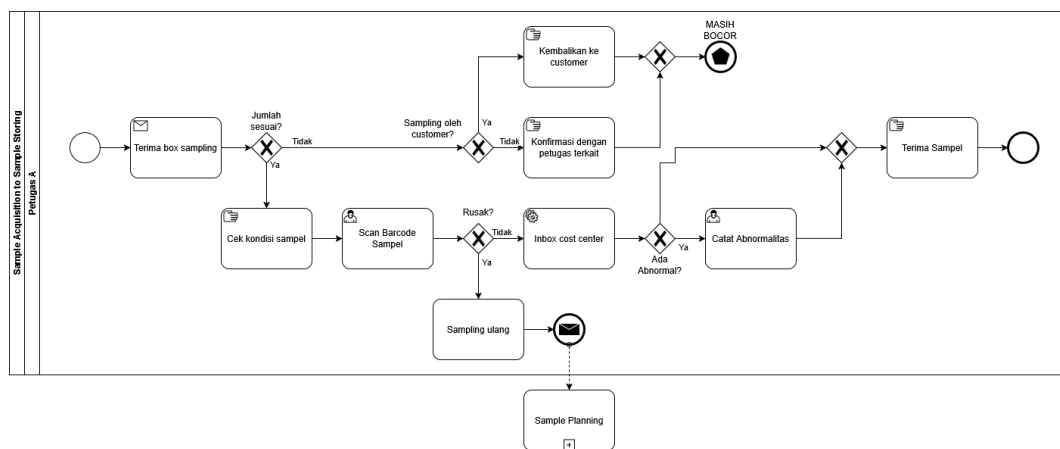
Gambar IV.3 BPMN alur *order* ke pengambilan sampel (to-be)

Alur ini dikelola oleh aktor Planning. *Sales Order* yang telah terkonfirmasi dipilih untuk dijadwalkan pengambilannya (UC-08). Planning menentukan titik-titik lokasi *sampling* yang relevan dengan *order* tersebut (UC-09), memilih teknisi

lapangan yang memiliki keahlian sesuai (UC-10), dan menetapkan tanggal pengambilan berdasarkan jadwal kosong seluruh teknisi terpilih (UC-11). Jika jadwal perlu diubah, sistem menyediakan alur *reschedule* (UC-12) yang mengembalikan proses ke pemilihan teknisi dan tanggal. Output alur ini adalah jadwal pengambilan yang terkunci dalam sistem, beserta identitas sampel dan titik *sampling* yang siap dieksekusi oleh Petugas Sampling.

IV.2.4 Alur Pengambilan Sampel ke Penyimpanan

Gambar IV.4 menggambarkan alur registrasi sampel yang masuk hingga penyimpanan sampel sebelum proses pengujian dimulai.

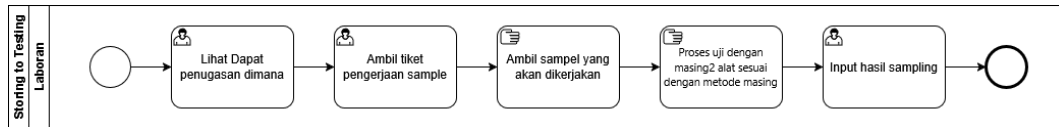


Gambar IV.4 BPMN alur pengambilan sampel ke penyimpanan (*to-be*)

Alur ini melibatkan dua aktor secara berurutan: Sales dan Petugas Sampling. Sebelum pengambilan dilaksanakan, Sales mengirimkan *barcode* dan label identifikasi sampel kepada klien (UC-14) beserta instruksi tata cara *sampling* (UC-15) melalui fitur *mailer* atau WhatsApp yang tersedia di sistem. Di lapangan, Petugas Sampling mencatat hasil pengukuran *in-situ* (UC-16) dan memberi keterangan abnormalitas jika ditemukan kondisi yang tidak normal pada titik pengambilan (UC-17). Sampel yang terkumpul kemudian dibawa ke laboratorium: penerimaan dilakukan dengan memindai *barcode* sampel untuk memperbarui statusnya (UC-18), dan setiap kondisi fisik sampel yang menyimpang dicatat sebagai *flag* di sistem (UC-19).

IV.2.5 Alur Penyimpanan ke Pengujian

Gambar IV.5 menggambarkan alur dari sampel yang tersimpan hingga proses pengujian parameter oleh analis dan pencatatan hasil uji.

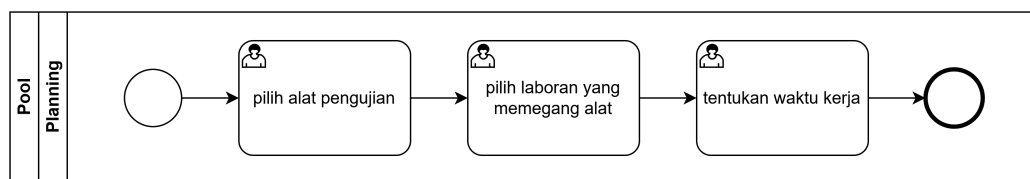


Gambar IV.5 BPMN alur penyimpanan ke pengujian (*to-be*)

Alur ini dimulai oleh Planning dan diselesaikan oleh Laboran. Planning menetapkan alat pengujian yang akan digunakan (UC-20), menugaskan laboran yang menguasai alat tersebut (UC-21), dan menentukan rentang waktu kerja pengujian (UC-22). Laboran kemudian melihat daftar penugasan yang masuk (UC-23), mengambil tiket pengerjaan dan sampel fisik dari penyimpanan (UC-24), lalu memasukkan hasil pengujian per parameter untuk setiap sampel (UC-25). Hasil yang tercatat menjadi input bagi alur penyeliaan — Penyelia memverifikasi setiap hasil sebelum sertifikat hasil uji dapat diterbitkan.

IV.2.6 Manajemen Jadwal Pengujian

Gambar IV.6 menggambarkan alur penjadwalan pengujian berdasarkan ketersediaan analis dan alat laboratorium dalam sistem Senling.



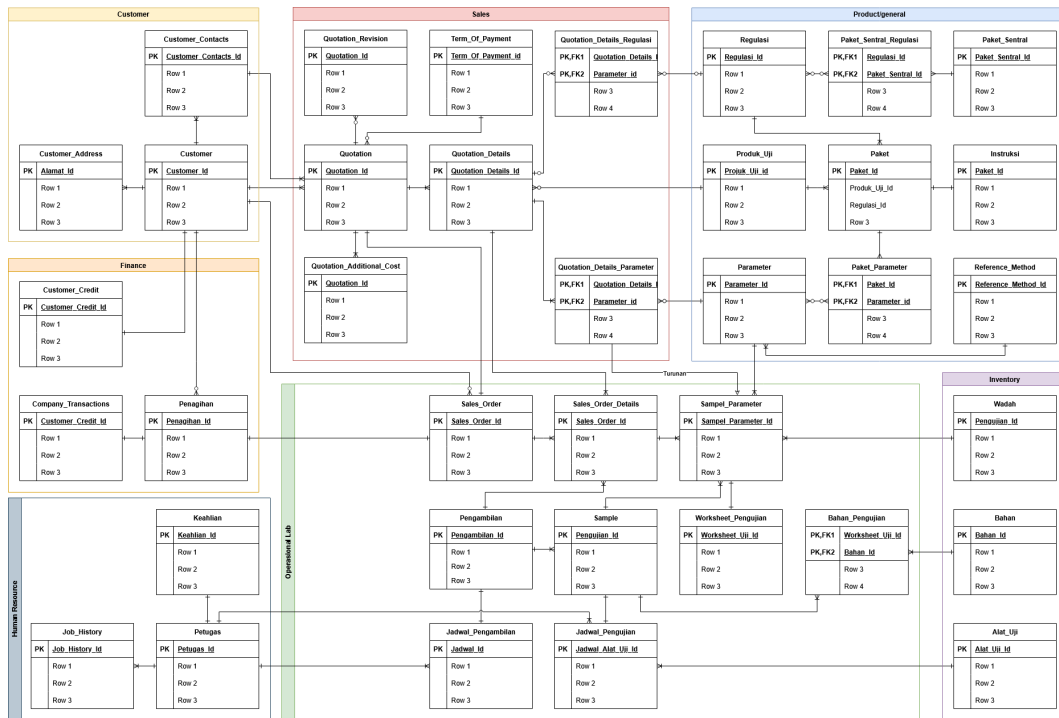
Gambar IV.6 BPMN manajemen jadwal pengujian (*to-be*)

Berbeda dari lima alur sebelumnya yang mengikuti siklus *Sales Order* tertentu, manajemen jadwal pengujian bersifat kapasitas dan dapat dilakukan secara proaktif tanpa menunggu *order* masuk. Planning menetapkan ketersediaan alat dan laboran untuk periode tertentu — ketersediaan ini disimpan sebagai data kapasitas yang siap dipasangkan (*matched*) saat pengujian perlu dijadwalkan (UC-20 s/d UC-22). Pemisahan antara jadwal kapasitas dan jadwal per-*order* membuat SSLab dapat merencanakan sumber daya laboratorium jauh sebelum sampel tiba, sekaligus mempertahankan fleksibilitas apabila ada perubahan mendadak seperti kerusakan alat atau ketidakhadiran laboran.

IV.3 Desain Data (*Entity Relationship Diagram*)

Gambar IV.7 menampilkan *entity relationship diagram* (ERD) Senling yang memperlihatkan pemisahan domain Customer, Product Knowledge, Sales, Operations,

dan Finance dalam satu basis data PostgreSQL.



Gambar IV.7 ERD sistem Senling

ERD Senling terdiri dari lima domain yang terisolasi: Customer (data klien dan kontak), Product Knowledge (produk uji, parameter, regulasi, alat, wadah), Sales (penawaran dan detailnya), Operations (sales order, pengambilan, sampel, hasil parameter), dan Finance (invoice, transaksi). Setiap domain membentuk kluster entitas yang mandiri dan hanya berinteraksi melalui relasi yang terdefinisi.

Domain Customer menyimpan data master klien SSLab dalam tiga entitas: profil perusahaan, data kontak penanggung jawab, dan daftar alamat. Domain ini menjadi titik referensi pertama dalam siklus penawaran — setiap quotation terhubung ke satu klien sebagai pemilik order. Verifikasi keberadaan klien di basis data merupakan langkah awal dalam alur Sales sebelum penawaran dapat dibuat.

Domain Product Knowledge menyimpan seluruh pengetahuan teknis laboratorium yang dibutuhkan untuk menyusun penawaran. Produk uji mendefinisikan matriks sampel yang tersedia, sementara regulasi mencatat standar yang berlaku untuk setiap produk uji. Parameter menyimpan item pengujian individual — masing-masing terhubung ke metode referensi, alat laboratorium, dan wadah sampel yang diperlukan untuk pelaksanaannya. Domain ini menjadi sumber referensi bagi domain Sales saat menyusun detail penawaran dan bagi domain Operations saat pengujian dilaksanakan.

Domain Sales mencatat siklus penawaran dari *draft* hingga konfirmasi klien.

Quotation menyimpan penawaran beserta status dan nomor PO yang disetujui; quotation detail menyimpan produk uji, titik *sampling*, dan frekuensi per baris penawaran; quotation detail parameter menyimpan parameter yang dipilih beserta harga satuannya. Saat klien mengonfirmasi dengan mengunggah PO, *QuotationService* membaca data domain Sales untuk menghasilkan *Sales Order* dan seluruh entitas turunannya di domain Operations dalam satu operasi atomik.

Domain Operations adalah klaster terbesar, mencakup siklus operasional laboratorium dari perencanaan hingga hasil pengujian. *Sales Order* dan detailnya menjadi titik masuk, diturunkan dari penawaran yang dikonfirmasi. Tabel pengambilan mencatat jadwal dan pelaksanaan pengambilan sampel di lapangan. Sampel merepresentasikan wadah fisik secara individual, sementara tabel *sample parameter* mencatat hasil pengujian per parameter untuk setiap sampel. Semua aktivitas lapangan dan laboratorium — dari perencanaan pengambilan hingga pencatatan hasil — bermuara pada domain ini.

Domain Finance mencatat kewajiban keuangan yang timbul dari *Sales Order* yang telah selesai dieksekusi. Entitas invoice dan transaksi direferensikan dari domain Operations melalui identitas *Sales Order*, tanpa ada modifikasi balik dari alur operasional ke Finance. Pemisahan ini memastikan bahwa pencatatan keuangan tidak bercampur dengan data operasional laboratorium dan dapat dikelola secara independen.

IV.4 Daftar Use Case

Sistem Senling mencakup 31 *use case* (UC-01 s/d UC-31) yang dikelompokkan ke dalam lima alur operasional sesuai domain bisnis SSLab. Pengelompokan ini mengikuti pembagian tanggung jawab antar aktor — Sales, Planning, Petugas Sampling, Laboran, dan Penyelia — serta urutan alur data dari penerimaan *order* hingga penerbitan sertifikat hasil uji. Tabel IV.1 menyajikan seluruh daftar *use case* beserta aktor dan status implementasinya.

Dari 31 *use case*, 15 (UC-01 s/d UC-15) telah diimplementasikan — mencakup seluruh alur penjualan dan perencanaan *sampling*. Sisa 16 *use case* (UC-16 s/d UC-31) merupakan target pengembangan fase berikutnya, meliputi alur pelaksanaan *sampling* di lapangan, penerimaan sampel, pengujian di laboratorium, dan penyeliaan hasil uji sebelum sertifikat diterbitkan.

Tabel IV.1 Daftar *use case* sistem Senling

Kode	Nama <i>Use Case</i>	Aktor	Status
Penjualan (Sales)			
UC-01	Generate <i>repeat order</i>	Sales	Implemented
UC-02	Cek <i>customer</i> di basis data	Sales	Implemented
UC-03	Input data <i>customer</i>	Sales	Implemented
UC-04	Buat <i>draft</i> penawaran	Sales	Implemented
UC-05	Buat penawaran	Sales	Implemented
UC-06	Revisi penawaran	Sales	Implemented
UC-07	<i>Upload Purchase Order</i>	Sales	Implemented
UC-14	Kirim <i>barcode</i> dan label ke klien	Sales	Implemented
UC-15	Kirim instruksi	Sales	Implemented
Perencanaan <i>Sampling</i> (Planning)			
UC-08	Pilih <i>Sales Order</i> yang akan di- <i>sampling</i>	Planning	Implemented
UC-09	Pilih titik <i>sampling</i>	Planning	Implemented
UC-10	Pilih teknisi	Planning	Implemented
UC-11	Pilih jadwal pengambilan	Planning	Implemented
UC-12	<i>Reschedule</i>	Planning	Implemented
UC-20	Pilih alat pengujian	Planning	Planned
UC-21	Pilih laboran pemegang alat	Planning	Planned
UC-22	Tentukan waktu kerja	Planning	Planned
Persiapan dan Pelaksanaan <i>Sampling</i>			
UC-13	Unduh <i>barcode</i> dan label	Petugas <i>Sampling</i>	Implemented
UC-16	Catat hasil pengukuran	Petugas <i>Sampling</i>	Planned
UC-17	Catat abnormalitas di lapangan	Petugas <i>Sampling</i>	Planned
UC-18	<i>Scan barcode</i> sampel	Petugas <i>Sampling</i>	Planned
UC-19	Catat abnormalitas penerimaan	Petugas <i>Sampling</i>	Planned
Pengujian di Laboratorium			
UC-23	Lihat penugasan	Laboran	Planned
UC-24	Ambil tiket dan sampel	Laboran	Planned
UC-25	Input hasil pengujian	Laboran	Planned
Penyeliaan			
UC-26	Pilih <i>Sales Order Detail</i>	Penyelia	Planned
UC-27	Pilih <i>Sales Order Detail</i> (tinjauan)	Penyelia	Planned
UC-28	Pilih parameter	Penyelia	Planned
UC-29	Kirim <i>draft</i> ke <i>customer</i>	Sales	Planned
UC-30	<i>Request</i> uji ulang	Penyelia	Planned
UC-31	<i>Request sampling</i> ulang	Penyelia	Planned

BAB V

RENCANA SELANJUTNYA

V.1 Metodologi Pengembangan

Metodologi pengembangan Senling mengikuti pendekatan iteratif yang berfokus pada penyampaian modul-modul inti secara bertahap. Setiap modul dirancang untuk mengatasi alur proses bisnis yang spesifik, divalidasi melalui pengujian fungsional, kemudian diintegrasikan ke dalam sistem monolitik. Metodologi ini didukung oleh sistem manajemen proyek yang berkembang dari pendekatan sederhana menjadi struktur yang lebih terorganisir seiring dengan pertumbuhan basis kode dan kompleksitas fitur.

V.1.1 Pendekatan Manajemen Proyek

Manajemen proyek pengembangan Senling mengalami evolusi metodologi untuk menyesuaikan dengan skala tim dan kompleksitas sistem. Fase awal menggunakan GitHub Issues sebagai sistem pelacakan tugas, namun pendekatan ini terbukti tidak optimal untuk tim dua orang karena kompleksitas pengelolaan dan transparansi konteks proyek. Pendekatan kemudian diubah menjadi sistem berbasis berkas Markdown (.md) sebagai pengganti GitHub Issues, memberikan fleksibilitas dan kesederhanaan dalam dokumentasi rencana pengembangan.

Seiring dengan peningkatan kompleksitas alur *Order to Acquisition* (fase implementasi kedua), tim mengadopsi disiplin perencanaan yang lebih ketat dengan memerlukan rencana tertulis sebelum implementasi dimulai. Praktik ini menghasilkan peningkatan signifikan dalam komunikasi tim, pengurangan *rework*, dan dokumentasi yang lebih konsisten. Pada fase-fase selanjutnya, metodologi berkembang lebih lanjut dengan mengintegrasikan alat-alat bantu pengembangan berbasis agen untuk otomatisasi aspek-aspek tertentu dari proses pengembangan, seperti pembuatan kerangka kode, verifikasi konsistensi skema basis data, dan dokumentasi otomatis.

V.1.2 Proses Pengembangan

Proses pengembangan sistem Senling mengikuti alur yang terdiri dari enam fase implementasi utama, masing-masing dipetakan langsung pada satu alur BPMN *to-be* yang telah dirancang pada Bab IV. Setiap fase mencakup tahapan analisis alur proses, perancangan model data, implementasi antarmuka pengguna dan layanan aplikasi, pengujian fungsional, dan integrasi dengan modul-modul sebelumnya.

Pengembangan dimulai dengan fase inisial berupa penyiapan dokumentasi dan infrastruktur selama dua minggu, diikuti dengan enam fase implementasi yang masing-masing menyelesaikan satu alur BPMN *to-be*. Durasi tiap fase berkisar antara satu hingga tiga minggu tergantung pada kompleksitas alur proses dan jumlah *use case* yang diimplementasikan. Alur *Order to Acquisition* mendapatkan alokasi tiga minggu karena merupakan alur perencanaan multistap yang paling kompleks, sekaligus menjadi titik transisi metodologi manajemen proyek dari pendekatan berbasis GitHub Issues ke pendekatan berbasis Markdown dengan dukungan agen.

V.1.3 Alat dan Infrastruktur Pengembangan

Pengembangan sistem Senling menggunakan *technology stack* berikut. Untuk aplikasi monolitik utama, digunakan Laravel 12 sebagai *framework* aplikasi web dan Filament 5 sebagai *framework* untuk membangun antarmuka administrasi yang cepat. Sistem basis data menggunakan PostgreSQL untuk penyimpanan data relasional dan Redis untuk *caching* serta manajemen sesi. Untuk penyajian aplikasi, digunakan Nginx sebagai *web server* dan *reverse proxy*.

Infrastruktur pengembangan dan pengujian awal (UAT) menggunakan Orange Pi 5 Pro sebagai server on-premise, memungkinkan tim untuk melakukan pengujian fungsional dalam lingkungan yang identik dengan rencana *deployment* produksi. Pendekatan ini mengurangi risiko *environment mismatch* dan memberikan validasi awal terhadap kelayakan perangkat keras yang dipilih. Selain itu, infrastruktur pengembangan lokal (mesin pengembang) didukung dengan Git untuk manajemen versi kode dan dokumentasi Markdown untuk pengelolaan proyek, memfasilitasi kolaborasi antara anggota tim.

V.1.4 Peran AI dalam Pengembangan

Pengembangan sistem Senling mengintegrasikan alat-alat berbasis agen dan *prompt caching* untuk meningkatkan efisiensi proses pengembangan, khususnya dalam otomatisasi tugas-tugas berulang dan pemeliharaan konsistensi dokumentasi. Alat-alat ini digunakan untuk fungsi-fungsi seperti verifikasi skema basis data, pembuatan

kerangka kode berdasarkan desain modul, dan pembaruan otomatis dokumentasi konteks proyek. Integrasi ini direncanakan untuk difasilitasi melalui Model Konteks Panjang yang memungkinkan agen untuk mempertahankan pemahaman mendalam tentang arsitektur sistem dan keputusan desain yang telah ditetapkan sebelumnya.

V.2 Rencana Pengembangan

Rencana pengembangan ini merupakan tindak lanjut dari desain konsep solusi yang telah dipaparkan pada Bab IV. Setiap fase implementasi dipetakan langsung pada satu alur BPMN *to-be* yang telah dirancang, sehingga pengembangan dapat dilakukan secara sekuensial mengikuti aliran data antar modul. Total durasi pengembangan diperkirakan 14 minggu, mencakup fase penyiapan dokumentasi dan infrastruktur selama 2 minggu serta enam fase implementasi yang masing-masing menyelesaikan satu alur proses bisnis.

V.2.1 Jadwal Pengembangan

Gambar V.1 menunjukkan visualisasi jadwal pengembangan untuk seluruh fase, diorganisasikan sesuai dengan urutan alur BPMN *to-be* pada Bab IV.

Fase	Minggu Pengembangan												
	1	2	3	4	5	6	7	8	9	10	11	12	13
F0: Dokumentasi & Infrastruktur	■	■											
F1: <i>Sales to Order</i>			■	■									
F2: <i>Order to Acquisition</i>					■	■	■						
F3: <i>Acquisition to Storing</i>								■	■				
F4: <i>Storing to Testing</i>										■	■		
F5: <i>Schedule Management</i>												■	
F6: Penyeliaan													■

Gambar V.1 Jadwal pengembangan sistem Senling

V.2.2 Detail Fase Pengembangan

V.2.2.1 Fase 0: Penyiapan Dokumentasi dan Infrastruktur (Minggu 1–2)

Fase awal berfokus pada penyiapan dokumentasi dan infrastruktur pengembangan. Aktivitas mencakup pemetaan proses bisnis saat ini (*as-is*), perancangan proses target (*to-be*) melalui BPMN, perancangan model data (ERD), dan identifikasi 31 *use case* sistem sebagaimana didokumentasikan pada Bab IV. Secara paralel, infrastruktur pengembangan disiapkan, mencakup konfigurasi Orange Pi 5 Pro, instalasi

stack teknologi (Laravel, PostgreSQL, Redis, Nginx), serta penyiapan repositori Git dan sistem dokumentasi berbasis Markdown sebagai pondasi manajemen proyek.

V.2.2.2 Fase 1: *Sales* ke *Order* (Minggu 3–4)

Fase ini mengimplementasikan alur BPMN *to-be Sales to Order* (lihat Bab IV Subbab 2.2) yang ditangani oleh aktor Sales. Implementasi mencakup verifikasi identitas klien, pendaftaran klien baru, penyusunan *draft* penawaran, penerbitan penawaran final, revisi penawaran, dan konfirmasi melalui unggahan *Purchase Order* (UC-01 hingga UC-07). Fase ini juga menyelesaikan migrasi modul Penawaran dari sistem CI3 lama ke arsitektur Laravel dan Filament, sehingga modul ini menjadi fondasi sistem yang memastikan integritas data dan kontinuitas bisnis.

V.2.2.3 Fase 2: *Order* ke *Acquisition* (Minggu 5–7)

Fase ini mengimplementasikan alur BPMN *to-be Order to Acquisition* (lihat Bab IV Subbab 2.3) yang ditangani oleh aktor Planning. Implementasi mencakup pemilihan *Sales Order* untuk dijadwalkan, pemilihan titik *sampling*, pemilihan teknisi, penetapan jadwal pengambilan, alur *reschedule*, dan pencetakan *barcode* serta label sampel (UC-08 hingga UC-13). Durasi 3 minggu untuk fase ini mencerminkan kompleksitas alur perencanaan multistap. Fase ini juga merupakan titik transisi metodologi manajemen proyek: pada minggu kelima tim beralih dari GitHub Issues ke sistem pelacakan berbasis Markdown; pada minggu keenam diterapkan disiplin perencanaan tertulis sebelum implementasi; pada minggu ketujuh diintegrasikan alat-alat bantu pengembangan berbasis agen untuk meningkatkan determinisme proses pengembangan.

V.2.2.4 Fase 3: *Acquisition* ke *Storing* (Minggu 8–9)

Fase ini mengimplementasikan alur BPMN *to-be Acquisition to Storing* (lihat Bab IV Subbab 2.4) yang melibatkan aktor Sales dan Petugas Sampling. Implementasi mencakup pengiriman *barcode* dan instruksi *sampling* kepada klien, pencatatan hasil pengukuran di lapangan, pencatatan abnormalitas di titik pengambilan, pemindaian *barcode* sampel saat penerimaan di laboratorium, dan dokumentasi abnormalitas kondisi fisik sampel (UC-14 hingga UC-19). Fase ini menjembatani aktivitas lapangan dengan operasi laboratorium.

V.2.2.5 Fase 4: *Storing* ke *Testing* (Minggu 10–11)

Fase ini mengimplementasikan alur BPMN *to-be Storing to Testing* (lihat Bab IV Subbab 2.5) yang dimulai oleh aktor Planning dan diselesaikan oleh Laboran. Implementasi mencakup penetapan alat pengujian, penugasan laboran pemegang alat, penentuan waktu kerja, tampilan daftar penugasan, pengambilan tiket dan sampel dari penyimpanan, serta pencatatan hasil pengujian per parameter (UC-20 hingga UC-25). Fase ini merupakan jantung operasi laboratorium dan memiliki ketergantungan data yang kompleks terhadap modul-modul sebelumnya.

V.2.2.6 Fase 5: Manajemen Jadwal Pengujian (Minggu 12)

Fase ini mengimplementasikan alur BPMN *to-be Schedule Management* (lihat Bab IV Subbab 2.6) yang bersifat kapasitas dan dapat dijalankan secara proaktif terlepas dari siklus *Sales Order* tertentu. Implementasi memungkinkan Planning untuk menetapkan ketersediaan alat dan laboran untuk periode tertentu sebagai data kapasitas yang akan dipasangkan saat pengujian perlu dijadwalkan. Durasi 1 minggu dialokasikan karena alur ini relatif sederhana dan menggunakan kembali entitas dan layanan yang telah dibangun pada Fase 4.

V.2.2.7 Fase 6: Penyeliaan (Minggu 13–14)

Fase terakhir mengimplementasikan alur penyeliaan hasil uji yang ditangani oleh aktor Penyelia. Implementasi mencakup tinjauan hasil per parameter, persetujuan atau penolakan hasil, permintaan uji ulang atau *sampling* ulang apabila ditemukan ketidaksesuaian, serta pengiriman *draft* sertifikat hasil uji kepada klien (UC-26 hingga UC-31). Fase ini menjamin bahwa setiap sertifikat hasil uji yang diterbitkan telah melalui mekanisme pengendalian kualitas sesuai dengan standar ISO/IEC 17025 yang telah dipegang SSLab.

V.2.3 Catatan Jadwal

Jadwal di atas bersifat estimasi dan dapat berubah sesuai kondisi pengembangan yang aktual. Beberapa faktor yang dapat mempengaruhi jadwal antara lain perubahan persyaratan proses bisnis dari SSLab, tantangan teknis yang tidak terduga, atau ketersediaan anggota tim yang terbatas. Apabila terjadi pergeseran jadwal, prioritas akan diberikan pada kualitas implementasi dan pengujian fungsional daripada kecepatan penyelesaian.

V.3 Analisis Risiko

Tabel V.1 menunjukkan identifikasi risiko utama yang mungkin terjadi selama implementasi dan pengembangan sistem ERP beserta strategi mitigasi yang akan diterapkan.

Tabel V.1 Analisis Risiko Implementasi Sistem ERP

Risiko	Frekuensi	Dampak	Strategi Mitigasi
Keterlambatan go-live atau produksi sistem	Tinggi	Sedang	Definisikan scope MVP yang jelas (7 modul inti); tandai fitur di luar scope sebagai backlog; gunakan versioning untuk rilis bertahap
Scope creep — permintaan modul atau fitur tambahan di luar 7 modul yang disepakati	Sedang	Tinggi	Dokumentasikan batasan masalah sebagai acuan; semua perubahan scope harus disetujui secara eksplisit; kelola backlog terpisah
Kehilangan atau korupsi data selama pengembangan	Rendah	Tinggi	Backup database otomatis sebelum setiap migrasi; gunakan transaction-safe migrations di Laravel; simpan snapshot konfigurasi secara berkala
Parameter uji belum tersedia untuk validasi modul hasil uji	Tinggi	Sedang	Gunakan data dummy untuk functional testing; tandai modul hasil uji sebagai pending UAT; jangan menghalangi modul lain
Kerusakan atau kegagalan hardware <i>Orange Pi</i>	Rendah	Tinggi	Backup konfigurasi dan data secara berkala; siapkan SBC cadangan atau fallback ke development di mesin lokal

Catatan: **Frekuensi** mengacu pada estimasi kemungkinan terjadinya risiko, sedangkan **Dampak** mengacu pada estimasi dampak terhadap keberhasilan implementasi sistem jika risiko tersebut terjadi (Rendah/Sedang/Tinggi).

DAFTAR PUSTAKA

- Álvarez Ariza, Jonathan, dan Heyson Baez. January 2022. “Understanding the role of single-board computers in engineering and computer science education: A systematic literature review”. *Computer Applications in Engineering Education* 30 (): 304–329. <https://doi.org/10.1002/cae.22439>.
- Arief, Lathifah, Fajril Akbar, Nefy Novani, dan Iqbal Saputra. September 2018. “Pengujian Kinerja Server Portable Berbasis Single Board Computer (SBC) Dalam Mendukung Kegiatan Pembelajaran”. *Jurnal Teknologi dan Sistem Informasi* 4 (): 98–106. <https://doi.org/10.25077/TEKNOSI.v4i2.2018.98-106>.
- Aroca, Rafael, dan Luiz Gonçalves. December 2012. “Towards green data centers: A comparison of x86 and ARM architectures power efficiency”. *Journal of Parallel and Distributed Computing* 72 (): 1770–1780. <https://doi.org/10.1016/j.jpdc.2012.08.005>.
- Blem, E., J. Menon, dan Karthikeyan Sankaralingam. February 2013. “Power struggles: Revisiting the RISC vs. CISC debate on contemporary ARM and x86 architectures”, 1–12. ISBN: 978-1-4673-5585-8. <https://doi.org/10.1109/HPCA.2013.6522302>.
- Boza, Andres, Llanos Cuenca, Raul Poler, dan Zenon Michaelides. 2015. “The interoperability force in the ERP field”. *Production Management and Engineering*, https://riunet.upv.es/bitstream/handle/10251/78847/FP_Aboza_Rev1210_June-copia.pdf.
- Cottafavi, Stefano. 2023. “Odoo - Free ERP on the Raspberry Pi”. Diakses 24 November 2025. <https://www.stefanocottafavi.com/embedded/odoo-raspberry/>.
- Forum, Frappe Community. 2023. “Setting Up ERPNext on a Raspberry Pi 5 Running Ubuntu 23.10”. Diakses 24 November 2025. <https://discuss.frappe.io/t/setting-up-erpnext-on-a-raspberry-pi-5-running-ubuntu-23-10/111399>.

- Fowler, Martin, dan James Lewis. 2014. "Microservices: a definition of this new architectural term". Diakses 2 April 2026. <https://martinfowler.com/articles/microservices.html>.
- Goldston, Justin. 2020. "The Evolution of ERP Systems: A Literature Review". *International Journal of Research* 50:1–18.
- Hasanah, Nur, Dhanar Intan Surya Saputra, dan Dewi Laela Hilyatin. 2024. "Enterprise Resource Planning Based Management Information System for Micro, Small and Medium Enterprises in Indonesia: A Systematic Literature Review". *Journal of Artificial Intelligence and Digital Business (RIGGS)* 2 (2): 36–42. <https://journal.ilmudata.co.id/index.php/RIGGS/article/download/224/219>.
- International Organization for Standardization. 2005. *ISO/IEC 17025:2005 – General requirements for the competence of testing and calibration laboratories*. ISO/IEC 17025:2005. Diakses 2 April 2026. Geneva, Switzerland: International Organization for Standardization. <https://www.iso.org/standard/39883.html>.
- . 2011. *ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. ISO/IEC 25010:2011. Diakses 22 November 2025. Geneva, Switzerland: International Organization for Standardization. <https://www.iso.org/standard/35733.html>.
- Ltd., Frappe Technologies Pvt. 2024. "Introduction - Documentation for Frappe Apps". Diakses 21 November 2025. <https://docs.frappe.io/erpnext/introduction>.
- Ltd., Raspberry Pi. 2024. "Raspberry Pi 5". Diakses 22 November 2025. <https://www.raspberrypi.com/products/raspberry-pi-5>.
- Maddodi, Gururaj, Slinger Jansen, dan Rolf Jong. March 2018. "Generating Workload for ERP Applications through End-User Organization Categorization using High Level Business Operation Data", 200–210. <https://doi.org/10.1145/3184407.3184432>.
- Monk, Ellen F., dan Bret J. Wagner. 2008. *Concepts in Enterprise Resource Planning*. 2nd. Boston, MA: Course Technology. ISBN: 978-0-619-21663-3.

- Odun-Ayo, Isaac, M. Ananya, Frank Agono, dan Rowland Goddy-Worlu. July 2018. "Cloud Computing Architecture: A Critical Analysis", 1–7. <https://doi.org/10.1109/ICCSA.2018.8439638>.
- Peppe8o. 2023. "Installing an ERP and CRM with Odoo on Raspberry Pi". Diakses 24 November 2025. <https://peppe8o.com/odoo-raspberry-pi/>.
- Putra, R. A., dan A. P Sujana. 2019. "Implementasi Cluster Server Pada Raspberry Pi Dengan Menggunakan Metode Load Balancing". *Komputika Jurnal Sistem Komputer* 8 (1): 37–43. <https://doi.org/10.34010/komputika.v8i1.1623>.
- S.A., Odoo. 2025. "Odoo Documentation". Diakses 21 November 2025. <https://www.odoo.com/documentation>.
- Sentral Sistem Laboratory. 2024. "Sentral Sistem Laboratory". Diakses 5 April 2026. <https://www.sentralsistemlaboratory.com/>.
- Shenzhen Xunlong Software Co., Limited. 2024. "Orange Pi 5 Pro". Diakses 22 November 2025. <http://www.orangepi.org/html/hardWare/computerAndMicrocontrollers/details/Orange-Pi-5-Pro.html>.
- Younus, Salman, Kamlesh Kumar, Irfan Ali, Asif Laghari, dan Asif Ali. June 2024. "Systematic Analysis of On Premise and Cloud Services". *International Journal of Cloud Computing* 13 (): 214–242. <https://doi.org/10.1504/IJCC.2024.10063641>.